B.TECH IN COMPUTER ENGINEERING • SEMESTER III

DATABASE MANAGEMENT SYSTEMS

Course Code: 2413CEC3T2 • 100-Marks End-Semester Exam Complete Guide

Comprehensive Academic Revision • Theory • Master Comparisons • Practical Queries & Solved Numericals

Category 1: Theory

Comprehensive 5-mark and 10-mark standard exam answers covering all 6 syllabus modules (Q1 – Q33). Includes formal definitions, clean ASCII architectural diagrams, bullet points, and real-world examples.

Category 2: Comparisons

29 master "Differentiate Between" tables covering every concept pair in DBMS (Tables 1 – 29). Structured with 4–8 parameters per table for maximum examination scoring.

Category 3: Practical & Code

Complete step-by-step solved problems: Relational Algebra queries, SQL scripts, Normalization & Closure algorithms, PL/SQL procedures, functions & triggers, and Precedence Graph conflict tests.

Document Highlights & Exam Focus

- **University Syllabus Parity:** 100% matched to 2413CEC3T2 Modules 1–6.
- **Solved End-Sem Exam:** All questions from Nov 2025 Paper solved.
- **Solved CA1 & CA2 Tests:** Full numericals, BCNF proofs, and SQL problems.
- **Academic Layout:** High-density, print-ready, professional reference guide.

Prepared for B.Tech Computer Engineering Undergraduates • Semester III Examination

100-Mark Examination Blueprint & Mark Distribution

The End-Semester examination for Database Management Systems (Course Code: 2413CEC3T2) is conducted for 100 marks over a duration of 3 Hours (180 minutes). The standard question paper structure is outlined below:

Question	Question Pattern	Total Marks	Focus Area & Syllabus Coverage
Q.1	Attempt ALL (5 × 4 Marks)	**20 Marks**	Compulsory core concepts: File vs DBMS, Relational Algebra selection/projection, SQL aggregate functions, Lossless vs Lossy decomposition, PL/SQL trigger advantages.
Q.2	Attempt ANY 4 of 6 (4 × 5 Marks)	**20 Marks**	Short theory & problems: EER Generalization/ Specialization, Student/Enroll relational algebra, Views in SQL, Functional dependency definition, PL/SQL architecture, Precedence graph conflict test.
Q.3	Attempt ANY 4 of 6 (4 × 5 Marks)	**20 Marks**	Short theory & queries: Data abstraction & independence, σ vs π relational algebra, DELETE vs TRUNCATE vs DROP, Attribute closure & candidate keys, PL/SQL features, ACID properties.
Q.4	Attempt ANY 2 of 3 (2 × 10 Marks)	**20 Marks**	Long answers / System design: 3-level DBMS architecture & data independence, Library Management System schema & RA queries, Student table SQL CRUD & TCL transaction rollback.
Q.5	Attempt ANY 2 of 3 (2 × 10 Marks)	**20 Marks**	Advanced topics & code: Normalization (1NF to BCNF) & anomalies, PL/SQL employee stored procedure with audit trigger, Serializability (Conflict vs View) & recovery protocols.

Examination Time Management Strategy (180 Minutes)

Section	Questions to Answer	Allocated Time	Recommended Writing Pace
Question 1	5 compulsory questions (4 marks each)	**30 minutes**	6 minutes per question (definition + key points + example)
Question 2	Best 4 of 6 questions (5 marks each)	**35 minutes**	~8.5 minutes per question (neat diagram/table required)
Question 3	Best 4 of 6 questions (5 marks each)	**35 minutes**	~8.5 minutes per question (closure step-by-step / SQL syntax)
Question 4	Best 2 of 3 questions (10 marks each)	**35 minutes**	~17 minutes per question (full schema/architecture + queries)
Question 5	Best 2 of 3 questions (10 marks each)	**35 minutes**	~17 minutes per question (BCNF step-by-step / full PL/SQL code)
Final Review	Verify roll number, question numbering & units	**10 minutes**	Check for missing SQL semicolons, graph cycles & keys

UNIVERSAL 100-MARK SCORING STRATEGY

- **For 4–5 Mark Questions:** State 1 exact definition • Provide 1 diagram or syntax box • Give 4–5 distinct bullet points • Conclude with 1 concrete example.
- **For 10 Mark Questions:** Begin with formal definition & architectural overview • Draw a clean, labeled box diagram • Detail all components • Provide step-by-step mathematical/code working • Highlight advantages and design trade-offs.
- **For Differences / Comparisons:** ALWAYS draw a structured comparison table with a designated "Parameter / Basis" column. Never write continuous comparison paragraphs.
- **For Code / Queries:** Always specify the table schema first, write neat capitalized SQL/PL-SQL keywords, and provide sample data or expected output.

Document Table of Contents

CATEGORY 1: THEORY QUESTIONS (5 & 10 MARKS)	
Module 1: Database System Concepts, 3-Schema Architecture, Data Models, DBA & ER/EER Modeling (Q1 – Q8)	**Page 4**
Module 2: Relational Data Model, Integrity Constraints, ER-to-Relational Mapping & Relational Algebra (Q9 – Q11)	**Page 9**
Module 3: Structured Query Language (SQL) • DDL, DML, DCL, TCL, Integrity Constraints, Views & Subqueries (Q12 – Q15)	**Page 12**
Module 4: Relational Database Design • Design Guidelines, Functional Dependencies, Normal Forms (1NF–BCNF) & Decomposition (Q16 – Q20)	**Page 15**
Module 5: Procedural Language/SQL (PL/SQL) • Features, Architecture, Blocks, Datatypes, Cursors, Triggers, Procedures & Functions (Q21 – Q26)	**Page 18**
Module 6: Transactions & Query Processing • States, ACID, Serializability, Concurrency (2PL), Recovery (Shadow Paging) & Optimization (Q27 – Q33)	**Page 23**
CATEGORY 2: MASTER COMPARISON TABLES ("DIFFERENTIATE BETWEEN")	
Core Foundations: File System vs DBMS • Physical vs Logical Data Independence • Strong vs Weak Entities (Tables 1 – 3)	**Page 28**
ER & Relational Models: Generalization vs Specialization vs Aggregation • Keys (Super, Candidate, Primary, Alternate, Foreign) (Tables 4 – 6)	**Page 29**
Relational Algebra & Joins: Unary vs Binary • σ vs π • Theta vs Equi vs Natural Join • Inner vs Outer Joins (Tables 7 – 10)	**Page 30**
SQL Commands & Clauses: DDL vs DML vs DCL vs TCL • DELETE vs TRUNCATE vs DROP • WHERE vs HAVING • Subquery Types (Tables 11 – 14)	**Page 31**
Normalization & Design: Dependencies (Partial, Full, Transitive) • 1NF vs 2NF vs 3NF vs BCNF • Lossless vs Lossy • Dependency Preservation (Tables 15 – 18)	**Page 33**
PL/SQL Constructs: SQL vs PL/SQL • Procedures vs Functions • Implicit vs Explicit Cursors • Row vs Statement Triggers • Triggers vs Procedures (Tables 19 – 23)	**Page 35**
Transactions & Recovery: Serial vs Serializable • Conflict vs View Serializability • S vs X Locks • 2PL Protocols • Log vs Shadow Paging • Deferred vs Immediate Update (Tables 24 – 29)	**Page 36**
CATEGORY 3: CODE, QUERIES & PRACTICAL NUMERICALS	
3.1 Relational Algebra Expressions: Solved Employee, Student, Course, and Library Management queries	**Page 39**
3.2 Relational Schema Design & Mapping: Full Library Management schema and ER-to-Relational mapping rules	**Page 40**
3.3 SQL Solved Queries & Scripts: Student CRUD & TCL transaction, branch analytics, aggregates, pattern matching, joins & subqueries	**Page 41**
3.4 Functional Dependencies & Normalization: Attribute Closure algorithm, Candidate Key solving, BCNF decomposition step-by-step	**Page 44**
3.5 PL/SQL Complete Code: Employee hire stored procedure + audit trigger, library fine function, explicit cursor bonus, weekend constraint trigger	**Page 47**
3.6 Transactions & Serializability: Precedence Graph conflict testing, topological sort, blind write view serializability	**Page 50**

CATEGORY 1: THEORY QUESTIONS (5 & 10 MARKS)

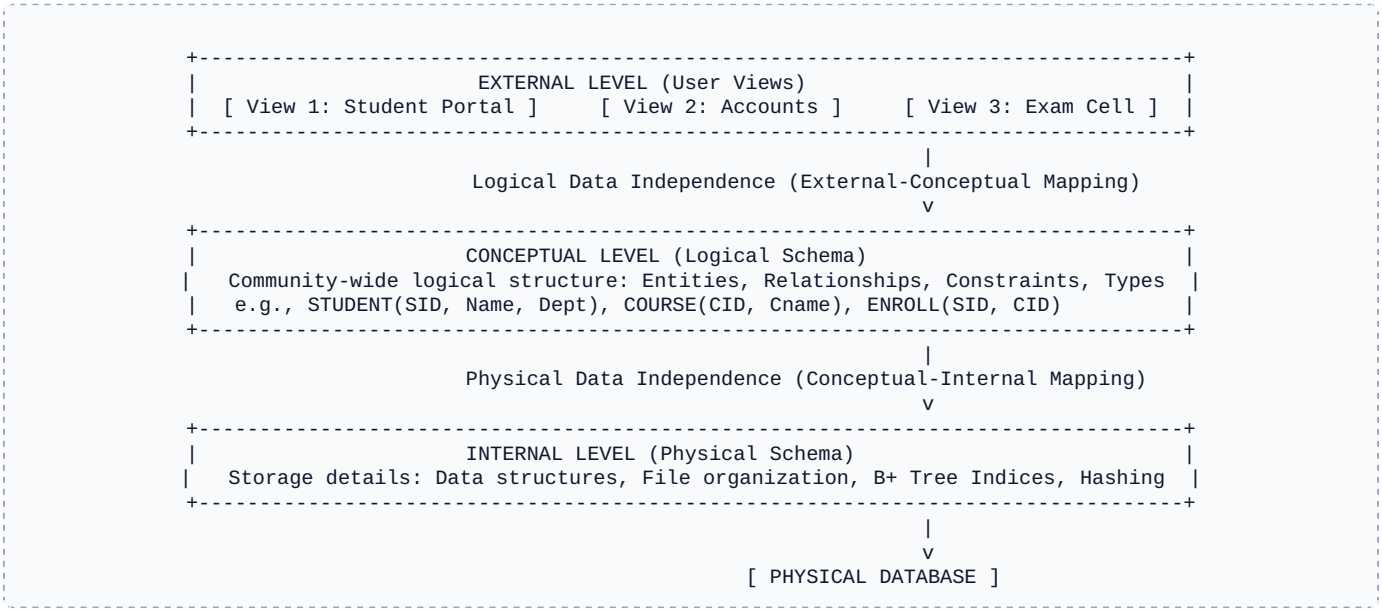
Structured academic answers strictly formatted for examination scoring. Each answer incorporates core definitions, architectural/conceptual diagrams, enumerated key points, and concrete examples.

Module 1: Database System Concepts and ER Model

Q1. Explain the Three-Level (Three-Schema) DBMS Architecture. Describe the role of Data Abstraction and Data Independence (Logical and Physical). 10 MARKS END-SEM Q4.a • CA1 Q7

CORE DEFINITION

The **Three-Schema Architecture** (proposed by ANSI/SPARC) divides the database system into three independent levels of abstraction to separate user applications from the physical database: **External (View) Level**, **Conceptual (Logical) Level**, and **Internal (Physical) Level**.



1. The Three Levels of Abstraction

- **External Level (View Level):** Closest to users. Describes only the part of the database relevant to a specific user group while hiding the rest. Multiple external views can exist simultaneously (e.g., students see grades; accounts staff see fees).
- **Conceptual Level (Logical Level):** Describes *what* data is stored in the database and the relationships among the data. It defines the entire database schema logically (entities, attributes, relationships, constraints, security checks) without revealing physical storage mechanisms.
- **Internal Level (Physical Level):** Closest to physical storage. Describes *how* data is actually stored on disks, detailing record formats, access paths, indexing techniques (B+ trees, hashing), compression, and encryption.

2. Data Abstraction

Data abstraction hides complex storage and implementation details from users. The three levels correspond directly to three levels of abstraction: **Physical Abstraction** (lowest level, physical structures), **Logical Abstraction** (middle level, entities and constraints), and **View Abstraction** (highest level, simplified user interfaces).

3. Data Independence

The primary objective of the three-schema architecture is achieving **Data Independence**: the capacity to change the schema at one level of the database system without having to change the schema at the next higher level.

- **Logical Data Independence:** The ability to modify the conceptual schema without having to alter external schemas or existing application programs.
  Example: Adding a new attribute (e.g., `Student_Aadhaar`) or adding a new relation to the conceptual schema does not affect existing student grade queries.
  Mechanism: Adjusted via **External-Conceptual Mapping**.

- **Physical Data Independence:** The ability to modify the internal schema without having to alter the conceptual (or external) schemas.
  Example: Switching from an unindexed sequential file to a B+ Tree index, changing disk block sizes, or moving database files across physical partitions does not change table definitions or user queries.
  Mechanism: Adjusted via **Conceptual-Internal Mapping**.

> **EXAM HIGH-SCORE TIP**
> Physical data independence is much easier to achieve than logical data independence because application programs are heavily tied to the logical structure of the data they access.

Q2. Compare and contrast File Processing Systems and DBMS. Explain the Core Characteristics of a DBMS. 5 MARKS END-SEM Q1.a • CA1 Q1

> **CONCEPT SUMMARY**
> A **File Processing System** stores data in separate flat files managed directly by the operating system, resulting in severe data redundancy, isolation, and lack of concurrent access. A **Database Management System (DBMS)** is a centralized software suite designed to define, store, manipulate, and secure shared data systematically.

Key Characteristics of a DBMS

1. **Self-Describing Nature:** A DBMS contains not only the database itself but also a complete definition/metadata stored in the **System Catalog (Data Dictionary)**. File systems lack metadata awareness.
2. **Insulation Between Programs and Data (Program-Data Independence):** In DBMS, data structure definitions are stored in the catalog separately from application programs. In file systems, file structures are hardcoded into application source code.
3. **Support for Multiple Views:** A DBMS allows different users to view distinct subsets of the database tailored to their functional requirements.
4. **Sharing of Data and Multi-User Transaction Processing:** Built-in concurrency control ensures multiple users can update data concurrently without leaving the database in an inconsistent state (ACID enforcement).
5. **Enforcement of Integrity Constraints:** DBMS automatically validates data rules (e.g., `PRIMARY KEY`, `FOREIGN KEY`, `CHECK`) directly at the database engine level.

Q3. Discuss the Roles and Responsibilities of the Database Administrator (DBA) and Classify Database Users. 5 MARKS CA1 Q8

1. Responsibilities of the Database Administrator (DBA)

- **Schema Definition:** Designing and creating the original database schema (conceptual, internal, and external) using DDL.
- **Storage Structure and Access-Path Definition:** Configuring physical disk storage, table spaces, partitioning, and primary/secondary indices to optimize query response times.
- **Schema and Physical-Organization Modification:** Altering schemas and indices to improve performance as database requirements evolve over time.
- **Granting Authorization for Data Access:** Regulating data access permissions via DCL commands (`GRANT`, `REVOKE`) to prevent unauthorized inspection or modification.
- **Routine Maintenance and Backup/Recovery:** Scheduling regular automated backups, monitoring available disk space, and executing disaster recovery protocols when hardware or software failures occur.
- **Performance Tuning:** Profiling slow queries and monitoring database throughput, buffer cache hit ratios, and lock contention.

2. Categories of Database Users

- **Naive / Parametric Users:** Unsophisticated end-users who interact with the database exclusively through pre-built graphical user interfaces and canned transactions (e.g., railway ticket reservation clerks, ATM users).
- **Sophisticated Users:** Engineers, scientists, and business analysts who interact with the database using interactive SQL query tools to execute ad-hoc, complex analytical queries without writing procedural programs.
- **Application Programmers:** Software developers who write programs in host languages (Java, C++, Python) embedding DML statements to build database applications.

- **Specialized Users:** Users who write specialized database applications that do not fit into traditional data processing frameworks (e.g., GIS systems, expert systems).

Q4. Classify and Explain the Primary Data Models Used in Database Management Systems. 5 MARKS

CA1 Q2

A **Data Model** is an integrated collection of concepts for describing data, data relationships, data semantics, and data constraints. Data models are classified based on their level of abstraction:

1. **Relational Data Model (Most Prevalent):** Represents all data and relationships using two-dimensional tables (called *relations*) consisting of rows (*tuples*) and columns (*attributes*). Simple, mathematically grounded in set theory and first-order predicate logic (e.g., PostgreSQL, MySQL, Oracle).
2. **Entity-Relationship (ER) Model (High-Level Conceptual):** Models real-world enterprises in terms of real-world objects called *entities* and associations among them called *relationships*. Widely used for conceptual database design before schema implementation.
3. **Hierarchical Data Model (Tree-Based):** Organizes records into tree-like structures where each child record has exactly one parent record (1:N relationship). Employs physical pointers; inflexible for many-to-many relationships (e.g., IBM IMS).
4. **Network Data Model (Graph-Based):** Represents data as record types linked by sets. A child (member) record can have multiple parent (owner) records, forming a general directed graph (M:N supported natively) (e.g., CODASYL DBTG).
5. **Object-Oriented Data Model:** Represents data as objects containing both state (attributes) and behavior (methods), supporting inheritance, encapsulation, and complex data types (e.g., ObjectDB).

Q5. Explain the Entity-Relationship (ER) Model. Classify All Types of Attributes and Keys with Standard ER Notation. 10 MARKS

CA1 Q5 • CA1 Q9

1. Core ER Model Concepts

- **Entity:** A real-world object or thing that is distinguishable from other objects (e.g., a specific student "Rahul", a course "DBMS"). Represented by a **Rectangle**.
- **Entity Set:** A collection of similar entities sharing the same properties (e.g., all students in a university).
- **Strong Entity Set:** An entity set that possesses a primary key attribute sufficient to uniquely identify each of its entities. Represented by a single **Rectangle**.
- **Weak Entity Set:** An entity set that does not have sufficient attributes to form a primary key. It depends on an *identifying (owner) entity set* via an *identifying relationship*. It is identified using the owner's primary key plus its own *partial key (discriminator)*. Represented by a **Double Rectangle**; its identifying relationship by a **Double Diamond**; its partial key by a **Dashed Underline**.

2. Comprehensive Classification of Attributes

Attribute Type	Definition & Characteristics	Example	ER Symbol
Simple (Atomic)	Cannot be divided into subparts; atomic.	`Age`, `Salary`	Single Ellipse
Composite	Can be subdivided into smaller components that form independent attributes.	`Name` → (First, Middle, Last); `Address` → (Street, City, PIN)	Ellipse connected to sub-ellipses
Single-Valued	Holds exactly one value for a specific entity instance.	`DOB`, `Aadhaar_No`	Single Ellipse
Multivalued	Can hold a set of values for the same entity instance.	`Phone_Numbers`, `Skills`	**Double Ellipse**
Stored	Physically saved in the database tables.	`Date_of_Birth`	Single Ellipse
Derived	Computed dynamically from other stored attributes; not stored physically.	`Age` = Current_Date − DOB	**Dashed Ellipse**
Key Attribute	Uniquely identifies each entity instance in an entity set.	`Roll_No`, `Employee_ID`	**Underlined Text**

3. Detailed Classification of Keys

- **Super Key (SK):** Any set of one or more attributes that taken collectively allows us to uniquely identify an entity in the entity set. (A super key may contain redundant attributes).
- **Candidate Key (CK):** A *minimal* super key (a super key from which no attribute can be removed without losing the uniqueness property).
- **Primary Key (PK):** The candidate key selected by the database designer as the principal means of identifying entities in an entity set. Must be strictly **NOT NULL** and unique.
- **Alternate (Secondary) Key:** Any candidate key that was not chosen as the primary key.
- **Foreign Key (FK):** An attribute (or set of attributes) in one relation that references the primary key of another relation, enforcing referential integrity.
- **Composite Key:** A key composed of two or more attributes needed together to uniquely identify a record.
- **Surrogate (Artificial) Key:** A system-generated numeric identifier (e.g., auto-incrementing ID) with no business meaning, created purely to act as a primary key.

Q6. Explain Enhanced ER (EER) Concepts: Generalization, Specialization, Aggregation, and Constraints.

10 MARKS END-SEM Q2.a • CA1 Q6

1. Generalization (Bottom-Up Synthesis)

The process of extracting common attributes and relationships from two or more lower-level entity sets to synthesize a generalized higher-level entity set.
Example: Combining lower-level entity sets `CAR` (attributes: License_No, Price, Max_Speed) and `TRUCK` (attributes: License_No, Price, Load_Capacity) into a higher-level entity set `VEHICLE` with shared attributes `{License_No, Price}`.

2. Specialization (Top-Down Decomposition)

The process of designating sub-groupings within an entity set that possess distinctive attributes or participate in distinct relationships not shared by all members of the entity set.
Example: Specializing the higher-level entity set `EMPLOYEE` into lower-level entity sets `SALARIED_EMPLOYEE` (attribute: Annual_Salary) and `HOURLY_EMPLOYEE` (attributes: Hourly_Rate, Hours_Worked).

3. Aggregation (Abstracting Relationships as Entities)

An abstraction through which relationships are treated as higher-level entities. In standard ER modeling, a relationship cannot participate directly in another relationship. Aggregation wraps a relationship and its participating entities into an aggregate entity that can then participate in another relationship.
Example: An employee works on a project using specific machinery. The relationship `WORKS_ON(Employee, Project)` is aggregated and then linked via `USES` to `MACHINERY`.

4. Constraints on Specialization / Generalization

- **Disjointness Constraint:**
  - **Disjoint (d):** An entity can belong to at most one lower-level entity set (e.g., an employee is either Salaried OR Hourly, but cannot be both).
  - **Overlapping (o):** An entity can belong to multiple lower-level entity sets simultaneously (e.g., a person in a university can be both a `STUDENT` and an `EMPLOYEE`).
- **Completeness (Participation) Constraint:**
  - **Total Specialization (Double Line):** Every entity in the superclass MUST belong to at least one subclass.
  - **Partial Specialization (Single Line):** An entity in the superclass does not have to belong to any subclass.

Q7. Provide Complete Conceptual ER System Designs for Standard Examination Case Studies.

10 MARKS CA1 Q4

Case Study 1: Library Management System

- **Entities & Attributes:**
  - `BOOK` (ISBN, Title, Edition, Year, Price) • Multivalued attribute: `Author`
  - `BOOK_COPY` (Copy_ID, Status, Rack_No) — *Weak Entity* dependent on `BOOK`
  - `MEMBER` (Member_ID, Name, Member_Type, Phone, Join_Date)
  - `PUBLISHER` (Pub_ID, PName, Address, Contact_No)

- **Relationships & Cardinality:**
  - `PUBLISHES` between `PUBLISHER` and `BOOK` (1:N — One publisher publishes many books).
  - `HAS_COPY` between `BOOK` and `BOOK_COPY` (1:N identifying relationship).
  - `ISSUES / BORROWS` between `MEMBER` and `BOOK_COPY` (M:N — with attributes: Issue_Date, Due_Date, Return_Date, Fine_Amount).

Case Study 2: Hospital Management System

- **Entities & Attributes:**
  - `PATIENT` (Patient_ID, Name, DOB, Gender, Phone, Address)
  - `DOCTOR` (Doctor_ID, Name, Specialization, Phone, Consultation_Fee)
  - `ROOM / BED` (Room_No, Room_Type, Daily_Charge)
  - `BILL` (Bill_ID, Bill_Date, Room_Charges, Doctor_Charges, Total_Amount)
- **Relationships & Cardinality:**
  - `TREATS / APPOINTMENT` between `DOCTOR` and `PATIENT` (M:N — with attributes: Appt_Date, Diagnosis, Prescription).
  - `ADMITTED_TO` between `PATIENT` and `ROOM` (N:1 — with attributes: Admission_Date, Discharge_Date).
  - `GENERATES` between `PATIENT` and `BILL` (1:N).

Case Study 3: University Management System

- **Entities & Attributes:**
  - `DEPARTMENT` (Dept_ID, DName, Office_Location)
  - `STUDENT` (Roll_No, Name, DOB, Gender, Year)
  - `INSTRUCTOR` (Instructor_ID, Name, Designation, Salary)
  - `COURSE` (Course_ID, Title, Credits)
- **Relationships:**
  - `BELONGS_TO`: `STUDENT` to `DEPARTMENT` (N:1).
  - `OFFERS`: `DEPARTMENT` to `COURSE` (1:N).
  - `ENROLLS`: `STUDENT` and `COURSE` (M:N — with attributes: Semester, Grade).
  - `TEACHES`: `INSTRUCTOR` and `COURSE` (1:N).

Module 2: Relational Model and Relational Algebra

Q8. Explain the Relational Data Model and Define Relation, Tuple, Attribute, Domain, Degree, Cardinality, Schema, and Instance. 5 MARKS CA1 Q12

CORE DEFINITION

The **Relational Data Model** represents database data as a collection of mathematically defined relations (two-dimensional tables). Each row represents a collection of related data values corresponding to a real-world entity or relationship.

Fundamental Relational Concepts

- **Relation (*R*):** A two-dimensional table consisting of rows and columns. Mathematically, a relation on domains D_1, D_2, …, D_n is a subset of the Cartesian product $D_1 \times D_2 \times \ldots \times D_n$.
- **Tuple:** A single row in a relation representing a specific record/entity instance.
- **Attribute:** A named column in a relation representing a property of an entity.
- **Domain (*dom(A)*):** The set of all permissible atomic values that an attribute *A* can take (e.g., domain of `Age` is integers between 1 and 120).
- **Degree (Arity):** The total number of attributes (columns) in a relation.
  Example: `STUDENT(RollNo, Name, Branch, GPA)` has a degree of 4.
- **Cardinality:** The total number of tuples (rows) currently stored in a relation. Cardinality changes dynamically as tuples are inserted or deleted.
- **Relational Schema ($R(A_1, A_2, \ldots, A_n)$):** The logical blueprint/design of a relation, specifying the relation name, its attributes, and their respective domains/constraints. It is static and rarely changes.
- **Relational Instance (*r(R)*):** The specific collection of tuples stored in the relation at a given moment in time (database snapshot). It is dynamic.

Q9. Explain the Four Core Relational Integrity Constraints with Examples. Detail Referential Integrity and Referential Actions. 5 MARKS

1. Domain Constraint

Specifies that the value of each attribute *A* in any tuple must be an atomic value from the physical domain *dom(A)*. Data type, format, and value ranges are checked (e.g., entering 'ABC' in an integer field violates domain integrity).

2. Key Constraint

Specifies that no two distinct tuples in any valid relation state can have identical values for the candidate key attribute(s). Formally, for any distinct tuples $t_1, t_2 \in r(R)$, $t_1[SK] \neq t_2[SK]$.

3. Entity Integrity Constraint

States that **no primary key value can be NULL**. Because primary keys are used to identify individual tuples, allowing NULL would mean we cannot identify that entity.

4. Referential Integrity Constraint

DEFINITION

Maintains consistency across two relations. A tuple in referencing relation R_1 that has a foreign key *FK* referencing primary key *PK* of referenced relation R_2 must either: (a) Match an existing primary key value in R_2, or (b) Be wholly `NULL` (if foreign key allows nulls).

Referential Actions on Deletion / Update of Referenced Tuples

When a tuple in the referenced (parent) table is deleted or updated, one of four integrity actions can be triggered:

1. **ON DELETE CASCADE:** Automatically deletes all child records in the referencing table that reference the deleted parent record.
2. **ON DELETE SET NULL:** Automatically sets the foreign key attributes of all referencing child records to `NULL`.
3. **ON DELETE SET DEFAULT:** Sets the foreign key attributes of child records to their predefined default value.

4. **ON DELETE RESTRICT / NO ACTION (Default):** Rejects the deletion of the parent record if any child record references it.

Q10. Explain the Complete Step-by-Step Algorithm for Mapping ER/EER Diagrams to Relational Schemas.

10 MARKS CA1 Q10 • CA1 Q13

Mapping a conceptual ER/EER diagram to a relational database follows a rigorous 7-step transformation algorithm:

Step 1: Mapping of Regular (Strong) Entity Types

For each strong entity type *E*, create a relation *R* containing all simple attributes of *E*. For composite attributes, include only their simple component attributes. Choose one key attribute as the Primary Key (*PK*).

Step 2: Mapping of Weak Entity Types

For each weak entity type *W* with owner entity *E*, create a relation *R*. Include all simple attributes of *W*. Add as foreign key attributes in *R* the primary key attributes of the owner relation *E*. The primary key of *R* is the composite of `{PK of Owner, Partial Key of W}`.

Step 3: Mapping of Binary 1:1 Relationship Types

Three potential approaches exist. The standard approach is the **Foreign Key Approach**: Choose one relation (preferably with total participation) and insert the primary key of the other relation as a foreign key attribute. Add any attributes of the 1:1 relationship to this relation.

Step 4: Mapping of Binary 1:N Relationship Types

Identify the relation *S* that represents the entity type at the **N-side (many side)** of the relationship. Include as foreign key in *S* the primary key of the relation *T* (the 1-side). Include any simple attributes of the relationship in *S*.

Step 5: Mapping of Binary M:N Relationship Types

For each M:N relationship type *R*, create a **new relation (cross-reference / join table)** *S*. Include as foreign key attributes in *S* the primary keys of both participating relations. The composite of these foreign keys forms the primary key of *S*. Include any descriptive attributes of the M:N relationship in *S*.

Step 6: Mapping of Multivalued Attributes

For each multivalued attribute *A* of entity *E*, create a **new relation** *R*. Include attribute *A* plus the primary key *K* of *E* as a foreign key referencing *E*. The primary key of *R* is the combination `{K, A}`.

Step 7: Mapping of EER Specialization / Generalization Hierarchies

- **Multiple Relations (Superclass and Subclasses):** Create one table for the superclass with its primary key and shared attributes. Create separate tables for each subclass containing the superclass primary key (as both PK and FK) plus subclass-specific attributes. (Best for general specialization).
- **Single Relation with Type Flag:** Create one consolidated table containing all attributes of superclass and all subclasses, plus a `Type` attribute to indicate the subclass. Subclass-specific columns will be `NULL` when not applicable. (Efficient when subclasses have few distinct attributes).

Q11. Explain Relational Algebra Operations. Differentiate Between Unary and Binary Operations and Define Set Operations.

5 MARKS END-SEM Q1.b • Q3.b • CA1 Q11 • CA1 Q14

> **DEFINITION**
> **Relational Algebra** is a procedural query language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.

1. Unary Relational Operations (Operate on a Single Relation)

- **Selection ($\sigma_{condition}(R)$):** Filters rows that satisfy a specified predicate (horizontal subsetting).
  Syntax: $\sigma_{Salary > 50000}(EMPLOYEE)$
- **Projection ($\pi_{attribute_list}(R)$):** Selects specified columns and removes duplicate rows (vertical subsetting).
  Syntax: $\pi_{Name, Salary}(EMPLOYEE)$
- **Rename ($\rho_{S(B_1, ..., B_n)}(R)$ or $\rho_S(R)$):** Renames a relation and/or its attributes to disambiguate column names during self-joins.

2. Binary Relational Set Operations (Require Union-Compatibility)

Two relations *R* and *S* are **Union-Compatible** if they have the same degree (same number of attributes) and corresponding attributes have identical domains.

- **Union (R ∪ S):** Returns a relation containing all tuples that appear in *R*, in *S*, or in both. Duplicates are eliminated.
- **Intersection (R ∩ S):** Returns tuples that appear in both *R* and *S*. ($R \cap S \equiv R - (R - S)$).
- **Set Difference (R − S):** Returns tuples that appear in *R* but do not appear in *S*. (Note: $R - S \neq S - R$).
- **Cartesian Product (R × S):** Combines every tuple of *R* with every tuple of *S*. If degree of *R* is n_1 and *S* is n_2, degree is $n_1 + n_2$. If cardinality of *R* is c_1 and *S* is c_2, cardinality is $c_1 \times c_2$.

3. Relational Join & Division Operations

- **Theta Join ($R \bowtie_\theta S$):** Cartesian product followed by a selection condition θ: $\sigma_\theta(R \times S)$.
- **Equijoin:** A theta join where the condition θ uses only equality (=).
- **Natural Join (R ⋈ S):** An equijoin on all common attribute names, projecting out duplicate join attributes.
- **Division Operator (R ÷ S):** Used for queries involving the quantifier "FOR ALL". Returns tuples in *R* that are associated with *every* tuple in *S*.

Module 3: Structured Query Language (SQL)

Q12. Classify SQL Commands into DDL, DML, DCL, TCL, and DQL with Definitions, Purpose, Syntax, and Examples. 5 MARKS CA2 Sec A Q1 • Q2

SQL commands are categorized into five functional subsets based on their operational target:

Category	Purpose & Characteristics	Standard Commands	Transaction Behavior
DDL (Data Definition Language)	Defines, alters, and deletes database schema structures and metadata.	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`	**Auto-commit** (Changes are permanent immediately; cannot be rolled back).
DML (Data Manipulation Language)	Manipulates data instances stored inside existing schema tables.	`INSERT`, `UPDATE`, `DELETE`, `MERGE`	**Transactional** (Can be committed or rolled back using TCL).
DQL (Data Query Language)	Queries and retrieves data rows from relations.	`SELECT`	Read-only (Does not alter state).
DCL (Data Control Language)	Manages database security, user permissions, and access privileges.	`GRANT`, `REVOKE`	Takes effect immediately across user sessions.
TCL (Transaction Control Language)	Manages transactions and ensures atomicity and consistency during DML execution.	`COMMIT`, `ROLLBACK`, `SAVEPOINT`	Finalizes or aborts pending transaction blocks.

Syntax & Practical Examples

```
-- DDL Example: Creating and altering table structure
CREATE TABLE Employee (
    EmpId NUMBER(5) PRIMARY KEY,
    EmpName VARCHAR2(50) NOT NULL,
    Salary NUMBER(10,2) CHECK (Salary > 0)
);
ALTER TABLE Employee ADD Department VARCHAR2(30);

-- DML Example: Modifying table data
INSERT INTO Employee VALUES (101, 'Aditya Sharma', 65000, 'IT');
UPDATE Employee SET Salary = Salary * 1.10 WHERE Department = 'IT';

-- TCL Example: Safeguarding data changes
SAVEPOINT sp1;
DELETE FROM Employee WHERE EmpId = 101;
ROLLBACK TO sp1; -- Undoes the delete operation!
COMMIT;          -- Makes all preceding changes permanent
```

Q13. Explain SQL Integrity Constraints at Column Level and Table Level. 5 MARKS

Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency.

Core Constraint Types

- `NOT NULL`: Enforces that a column cannot contain `NULL` values (must be specified at **column level**).
- `UNIQUE`: Guarantees that all values in a column are distinct. Allows multiple `NULL` values (since `NULL != NULL` in SQL standard).
- `PRIMARY KEY`: Combination of `NOT NULL` and `UNIQUE`. Uniquely identifies each record. Exactly one primary key is permitted per table.
- `FOREIGN KEY`: Enforces referential integrity between two tables. Matches primary key of another table.
- `CHECK`: Enforces a condition that each value in the column must satisfy (e.g., `CHECK (Age >= 18)`).

- `DEFAULT`: Supplies a default value when no value is provided during an insert.

Column-Level vs Table-Level Syntax

```
-- Comprehensive Table Creation demonstrating both syntaxes
CREATE TABLE Student_Enrollment (
    RollNo NUMBER(6),                          -- Defined without constraint here
    CourseId VARCHAR2(10),
    EnrollDate DATE DEFAULT SYSDATE,           -- Column-level DEFAULT
    Marks NUMBER(5,2) CHECK (Marks BETWEEN 0 AND 100), -- Column-level CHECK
    Email VARCHAR2(100) NOT NULL,              -- Column-level NOT NULL

    -- Table-Level Constraints (Mandatory for Composite Keys and Cross-Column Checks)
    CONSTRAINT pk_enroll PRIMARY KEY (RollNo, CourseId),
    CONSTRAINT fk_student FOREIGN KEY (RollNo) REFERENCES Student(RollNo) ON DELETE CASCADE,
    CONSTRAINT uq_email UNIQUE (Email)
);
```

Q14. Evaluate the Usefulness of Views in SQL. What are their Advantages, and When Can a View Be Updated? 5 MARKS END-SEM Q2.c • CA2 Sec A Q5

> **DEFINITION**
> A **View** is a virtual table derived from the result-set of an underlying SQL query. A view does not store data physically on disk (except materialized views); only its query definition is stored in the system catalog.

Advantages and Usefulness of Views

1. **Data Security & Confidentiality:** Restricts user access to specific columns and rows, hiding sensitive information (e.g., exposing employee names and departments without exposing their salaries).
2. **Query Simplification:** Hides complex multi-table joins, aggregations, and subqueries behind a simple virtual relation name.
3. **Logical Data Independence:** If the underlying base table structure changes (e.g., splitting a table into two), a view can re-synthesize the original schema so application queries do not break.
4. **Data Customization:** Enables multiple concurrent users to see tailored data without duplicating physical storage.

Conditions for Updatable Views (DML Allowed on Views)

A view is **updatable** (allowing `INSERT`, `UPDATE`, `DELETE` to propagate to underlying base tables) only if all of the following conditions are met:

- The view is defined on **exactly one base table**.
- The `SELECT` clause does NOT contain the `DISTINCT` keyword.
- The query does NOT contain **Aggregate Functions** (`SUM`, `AVG`, `COUNT`, `MAX`, `MIN`).
- The query does NOT contain `GROUP BY` or `HAVING` clauses.
- The query does NOT use set operations (`UNION`, `INTERSECT`, `MINUS`).
- The view includes all `NOT NULL` columns of the base table that have no default values.

```
-- Creating an Updatable View with CHECK OPTION
CREATE VIEW CS_Student_View AS
SELECT RollNo, Name, Branch, GPA
FROM Student
WHERE Branch = 'Computer Science'
WITH CHECK OPTION; -- Prevents inserting rows where Branch != 'Computer Science'
```

Q15. Explain Subqueries / Nested Queries in SQL. Differentiate Single-Row, Multi-Row, and Correlated Subqueries. 5 MARKS CA2 Sec A Q5

> **DEFINITION**
> A **Subquery (Nested Query)** is an SQL query nested inside another `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement. The inner query executes and passes its result to the outer query.

1. Single-Row Subqueries

Returns at most one row and one column (a scalar value). Evaluated using single-row comparison operators (`=` , `>` , `<` , `>=` , `<=` , `<>`).

```
-- Find students whose marks are higher than the class average
SELECT Name, Marks FROM Student
WHERE Marks > (SELECT AVG(Marks) FROM Student);
```

2. Multi-Row Subqueries

Returns multiple rows. Requires multi-row operators:

- `IN` : Matches any value in the returned list.
- `ANY / SOME` : Compares a value to each value in the list (e.g., `> ANY(...)` means greater than the minimum).
- `ALL` : Compares a value to all values in the list (e.g., `> ALL(...)` means greater than the maximum).

```
-- Find employees whose salary is higher than ALL employees in the 'Sales' department
SELECT EmpName, Salary FROM Employee
WHERE Salary > ALL (SELECT Salary FROM Employee WHERE Department = 'Sales');
```

3. Correlated Subqueries

KEY MECHANISM

Unlike independent subqueries (which execute once before the outer query), a **Correlated Subquery** references columns from the outer query relation. It executes **once for every candidate row** evaluated by the outer query. Uses `EXISTS` / `NOT EXISTS` .

```
-- Find employees who earn more than the average salary of THEIR OWN department
SELECT E.EmpName, E.Salary, E.Department
FROM Employee E
WHERE E.Salary > (
    SELECT AVG(D.Salary)
    FROM Employee D
    WHERE D.Department = E.Department -- Correlated link!
);
```

Module 4: Relational Database Design (Normalization)

Q16. Discuss the Informal Design Guidelines for Relational Schemas. Explain Why Normalization is Required. 5 MARKS CA2 Sec B Q2

Four informal guidelines dictate sound relational database design to prevent storage and manipulation problems:

1. **Guideline 1 — Semantics of Attributes:** Design relation schemas so that each relation corresponds to a single entity type or relationship type. Attributes from distinct entity types (e.g., student and department) should not be mixed indiscriminately in a single table.
2. **Guideline 2 — Reducing Redundant Information:** Minimize storage redundancy to avoid update, insertion, and deletion anomalies.
3. **Guideline 3 — Reducing NULL Values:** Avoid placing attributes in base relations that frequently take `NULL` values. Null values waste physical storage, complicate aggregate calculations (`COUNT` , `AVG`), and require three-valued logic. (Instead, place sparsely populated attributes in a separate relation with the primary key).
4. **Guideline 4 — Disallowing Spurious Tuples:** Design relations such that when natural joins are performed over relations with common attributes, no invalid or "spurious" tuples are generated. Formally, relations must satisfy the **Lossless-Join property**.

Q17. Define Functional Dependency. Explain Trivial vs Non-Trivial FDs and Armstrong's Axioms. 5 MARKS END-SEM Q2.d • CA2 Sec B Q1

FORMAL DEFINITION

A **Functional Dependency (FD)** denoted by $X \rightarrow Y$ between two sets of attributes X and Y in relation schema R specifies a constraint on the possible tuples that can form any relation state $r(R)$.
Formally: For any two tuples $t_1, t_2 \in r(R)$, if $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$. We say "X functionally determines Y" or "Y is functionally dependent on X".

Trivial vs Non-Trivial Functional Dependencies

- **Trivial FD:** An FD $X \rightarrow Y$ is trivial if $Y \subseteq X$ (the right-hand side is a subset of the left-hand side).
  Example: {RollNo, Name} → RollNo is trivial because RollNo ∈ {RollNo, Name}. It conveys no new constraints.
- **Non-Trivial FD:** An FD $X \rightarrow Y$ is non-trivial if $Y \not\subseteq X$. It is *completely non-trivial* if $X \cap Y = \emptyset$ (e.g., RollNo → Name).

Armstrong's Axioms (Sound and Complete Inference Rules)

PRIMARY AXIOMS

1. **Reflexivity Rule:** If $Y \subseteq X$, then $X \rightarrow Y$.
2. **Augmentation Rule:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any set of attributes Z.
3. **Transitivity Rule:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Secondary (Derived) Inference Rules

- **Union (Additive) Rule:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- **Decomposition (Projective) Rule:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
- **Pseudotransitivity Rule:** If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$.

Q18. Explain the Need for Normalization. Illustrate Insertion, Deletion, and Modification Anomalies with an Example. 5 MARKS END-SEM Q5.a • CA2 Sec B Q2

Normalization is the systematic process of decomposing unnormalized relations to eliminate data redundancy and prevent data anomalies while preserving data integrity and functional dependencies.

Illustrative Bad Relation: `EMP_DEPT(EmpId, EmpName, DeptId, DeptName, DeptManager)`

EmpId	EmpName	DeptId	DeptName	DeptManager
101	Alice	D01	Research	Dr. Stone
102	Bob	D01	Research	Dr. Stone

EmpId	EmpName	DeptId	DeptName	DeptManager
103	Charlie	D02	Marketing	Ms. Vance

The Three Database Anomalies

1. **Insertion Anomaly:** We cannot insert a new department (e.g., DeptId = 'D03', DeptName = 'Finance') unless we have an employee assigned to it, because `EmpId` is the primary key and cannot be `NULL` (violates Entity Integrity).
2. **Deletion Anomaly:** If employee 103 (Charlie) leaves the company and we delete his record, we inadvertently lose all information about department 'D02' (Marketing and Ms. Vance) because it was the only record holding that department's details!
3. **Update (Modification) Anomaly:** If the manager of department 'D01' changes from 'Dr. Stone' to 'Dr. Banner', we must update multiple records. If we update Alice's row but miss Bob's row, the database enters an inconsistent state where department 'D01' appears to have two different managers.

Q19. Comprehensive Guide to Normal Forms: 1NF, 2NF, 3NF, and BCNF. Provide Definitions, Conditions, and Conversion Steps. 10 MARKS END-SEM Q5.a • CA2 Sec B Q3

```
                [ UNNORMALIZED ] --- (Remove Multivalued/Composite Attributes) ---> [ 1NF ]
                                                                                      |
                 [ 1NF ] ------------ (Remove Partial Dependencies) --------------> [ 2NF ]
                                                                                      |
                 [ 2NF ] ------------ (Remove Transitive Dependencies) -----------> [ 3NF ]
                                                                                      |
              [ 3NF ] ------------ (Remove Non-Trivial FDs where LHS != Super Key) -> [ BCNF ]
```

1. First Normal Form (1NF)

- **Condition:** A relation *R* is in 1NF if and only if all attribute values are **atomic (indivisible)** and there are no repeating groups or multivalued attributes.
- **Violation:** A column holding multiple phone numbers (e.g., `{9876543210, 9123456780}`).
- **Resolution:** Create separate tuples for each multivalued item, or move the multivalued attribute into a new table with the original primary key.

2. Second Normal Form (2NF)

- **Condition:** A relation *R* is in 2NF if it is in 1NF and **no non-prime attribute is partially dependent on any candidate key**.
  Full Functional Dependency: Every non-prime attribute must depend on the *whole* candidate key, not a proper subset of it.
- **Violation:** In *R(StudentID, CourseID, StudentName, CourseFee)*, the Candidate Key is {StudentID, CourseID}.
  StudentID → StudentName is a **partial dependency** because StudentID is a proper subset of the composite key.
- **Resolution:** Decompose into R_1*(StudentID, StudentName)* and R_2*(StudentID, CourseID, CourseFee)*.

3. Third Normal Form (3NF)

- **Condition:** A relation *R* is in 3NF if it is in 2NF and **no non-prime attribute is transitively dependent on a candidate key**.
  Formal Test: For every non-trivial functional dependency *X → Y* in *R*, either:
  1. *X* is a **Super Key**, OR
  2. *Y* is a **Prime Attribute** (each attribute in *Y* is a member of some candidate key).
- **Violation:** In *R(EmpId, DeptId, DeptName)* with Candidate Key EmpId:
  EmpId → DeptId and *DeptId → DeptName* ⇒ *EmpId → DeptName* is a transitive dependency through non-key attribute DeptId.
- **Resolution:** Decompose into R_1*(EmpId, DeptId)* and R_2*(DeptId, DeptName)*.

4. Boyce-Codd Normal Form (BCNF / Strict 3NF)

- **Condition:** A relation *R* is in BCNF if and only if for every non-trivial functional dependency *X → Y*, ***X* must be a Super Key**.
  (BCNF eliminates the relaxation allowed in 3NF where *Y* could be a prime attribute).
- **When 3NF is NOT BCNF:** When a table has multiple overlapping composite candidate keys.
  Example: R(Student, Subject, Teacher) with FDs:

1. {Student, Subject} → Teacher
2. Teacher → Subject

Candidate Keys are {Student, Subject} and {Student, Teacher}.
In *Teacher → Subject*, Subject is a prime attribute, so the relation is in **3NF**. However, Teacher is NOT a super key, so it **violates BCNF**!

- **Resolution:** Decompose into R_1*(Teacher, Subject)* and R_2*(Student, Teacher)*.

Q20. Explain Relational Decompositions: Lossless-Join Decomposition and Dependency Preservation.

10 MARKS END-SEM Q1.d • CA2 Sec B Q5 • Q6

1. Lossless-Join (Non-Additive) Decomposition

FORMAL CONDITION FOR BINARY DECOMPOSITION

A decomposition of relation R into R_1 and R_2 is **lossless** with respect to a set of FDs F if and only if the natural join of the decomposed relations reconstructs the exact original relation without producing spurious tuples:

$$R_1 \bowtie R_2 = R$$

Mathematical Test: At least one of the following functional dependencies must be in the closure F^+:
1. $(R_1 \cap R_2) \rightarrow R_1$ (The common attributes form a super key for R_1), OR
2. $(R_1 \cap R_2) \rightarrow R_2$ (The common attributes form a super key for R_2).

If neither holds, the decomposition is **Lossy**, generating fake rows that corrupt data integrity upon rejoining.

2. Dependency-Preserving Decomposition

DEFINITION

A decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of relation R is **dependency-preserving** with respect to F if the union of the projections of F on each decomposed relation R_i logically implies all dependencies in F:

$$(F_1 \cup F_2 \cup \ldots \cup F_m)^+ = F^+$$

- **Significance:** When a decomposition is dependency-preserving, the DBMS can enforce all functional dependencies locally within individual tables without executing expensive cross-table joins during every `INSERT` or `UPDATE`.
- **Trade-off:** It is always possible to find a 3NF decomposition that is both lossless-join AND dependency-preserving. However, a BCNF decomposition is always guaranteed to be lossless, but it may *not* always preserve all functional dependencies!

Module 5: Procedural Language/SQL (PL/SQL)

Q21. Explain the Main Features and Advantages of PL/SQL over Standard SQL. 5 MARKS

END-SEM Q1.e • Q3.e

> **DEFINITION**
> **PL/SQL (Procedural Language/SQL)** is Oracle Corporation's procedural extension to SQL. It combines the declarative data retrieval power of SQL with standard procedural programming constructs such as conditional branching (`IF-THEN-ELSE`), iterative loops (`FOR` , `WHILE`), modularity (procedures, functions, packages), and robust exception handling.

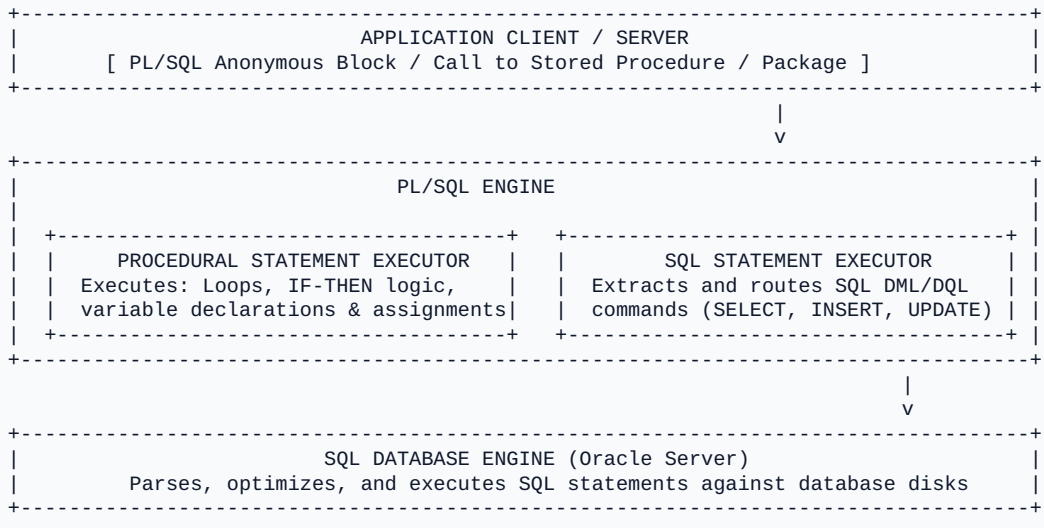
Main Features of PL/SQL

1. **Block-Structured Language:** Code is organized into modular units called *blocks* (anonymous blocks, procedures, functions, packages, triggers) which can be nested.
2. **Seamless SQL Integration:** Direct support for standard SQL DML commands (`SELECT INTO` , `INSERT` , `UPDATE` , `DELETE`) and transaction controls (`COMMIT` , `ROLLBACK`) without complex API wrappers.
3. **Procedural Flow Control:** Full support for conditional execution (`IF-THEN-ELSIF-ELSE` , `CASE`) and looping structures (Basic `LOOP` , `WHILE` , Numeric `FOR` , Cursor `FOR`).
4. **Exception Handling:** Built-in mechanism to catch and resolve runtime errors gracefully without crashing applications.
5. **Anchored Data Types:** Supports `%TYPE` (inherits column data type) and `%ROWTYPE` (inherits entire table record structure) ensuring code remains maintainable when schema alters.

Advantages of PL/SQL over Standalone SQL

- **Drastic Reduction in Network Traffic:** With standard SQL, applications send queries to the server one by one over the network, incurring heavy round-trip latency. In PL/SQL, an entire block containing dozens of statements is transmitted to the database server in a single network round-trip and processed locally in bulk.
- **Superior Performance:** Compiled PL/SQL stored procedures reside in the shared pool memory of the server, eliminating repetitive compilation overhead.
- **Enhanced Security & Maintainability:** Business logic is centralized on the secure server rather than duplicated across multiple client applications.

Q22. Explain the Architecture of PL/SQL and How It Enables Procedural Execution within an SQL Environment. 5 MARKS END-SEM Q2.e



Architectural Workflow

- When an application transmits a PL/SQL block, the block enters the **PL/SQL Engine** (which can reside inside the database server or within client-side development tools like Oracle Forms).

- The PL/SQL engine analyzes the block:
  - **Procedural statements** (assignments, loops, branch logic, calculations) are executed directly by the internal **Procedural Statement Executor**.
  - **SQL statements** (queries, DML operations) are separated and dispatched to the **SQL Statement Executor**, which forwards them to the Oracle **SQL Database Engine**.
- This architecture ensures high efficiency: procedural computations execute at high speed in memory without crossing query engine boundaries, while data access operations leverage the full query optimizer.

Q23. Describe the Structure of a PL/SQL Block, Identifiers, and Anchored Data Types (%TYPE, %ROWTYPE). 5 MARKS

1. Fundamental Block Structure

```
DECLARE
    -- Optional: Declaration section for variables, constants, cursors, user exceptions
BEGIN
    -- Mandatory: Executable statements section (procedural logic and SQL DML)
EXCEPTION
    -- Optional: Exception handling section for runtime errors
END;
/
```

2. Anchored Data Types

- `%TYPE` **Attribute:** Declares a variable with the exact data type and precision of a previously defined database column or another variable.
  Syntax: `v_emp_salary Employee.Salary%TYPE;`
  Advantage: If the DBA alters `Employee.Salary` from `NUMBER(8,2)` to `NUMBER(10,2)`, the PL/SQL code automatically adapts without requiring manual code changes!
- `%ROWTYPE` **Attribute:** Declares a composite record variable representing an entire row of a table or view.
  Syntax: `v_emp_record Employee%ROWTYPE;`
  Access: Fields are accessed using dot notation: `v_emp_record.EmpName`, `v_emp_record.Salary`.

Q24. Explain Cursors in PL/SQL. Differentiate Implicit vs Explicit Cursors, Detail the Cursor Lifecycle and Cursor Attributes. 10 MARKS

> **DEFINITION**
> A **Cursor** is a pointer to a private work area in memory (Context Area) allocated by the Oracle server to hold and process the result set of an SQL query. Cursors enable procedural languages to iterate through a multi-row query result row-by-row.

1. Implicit Cursors

Automatically declared and managed by Oracle for all single-row DML operations (`INSERT`, `UPDATE`, `DELETE`) and single-row `SELECT INTO` queries. Accessed using the prefix `SQL%` (e.g., `SQL%ROWCOUNT`, `SQL%FOUND`).

2. Explicit Cursors (User-Defined Multi-Row Cursors)

Programmer-declared cursors for processing multi-row queries. They follow a strict 4-step lifecycle:

1. **DECLARE:** Names the cursor and defines the associated SQL query structure.
   `CURSOR cur_emp IS SELECT EmpId, EmpName, Salary FROM Employee WHERE Department = 'IT';`
2. **OPEN:** Allocates private context memory, parses/evaluates the query, binds variables, and identifies the active result set. The pointer sits before the first row.
   `OPEN cur_emp;`
3. **FETCH:** Retrieves the current row values into PL/SQL variables and advances the pointer to the next row.
   `FETCH cur_emp INTO v_id, v_name, v_sal;`
4. **CLOSE:** Releases context memory back to the database server.
   `CLOSE cur_emp;`

3. The Four Cursor Attributes

Attribute	Meaning for Explicit Cursor (`cur%`)	Meaning for Implicit Cursor (`SQL%`)
`%FOUND`	Returns `TRUE` if the most recent fetch returned a row; `FALSE` otherwise.	Returns `TRUE` if an insert, update, or delete affected at least one row.
`%NOTFOUND`	Returns `TRUE` if the most recent fetch failed to return a row (end of data).	Returns `TRUE` if the DML statement affected zero rows.
`%ISOPEN`	Returns `TRUE` if the cursor is currently open in memory.	Always evaluates to `FALSE` (Oracle closes implicit cursors immediately).
`%ROWCOUNT`	Returns the cumulative number of rows fetched so far.	Returns the total number of rows affected by the DML statement.

Q25. Explain Database Triggers in PL/SQL. Discuss Types, Execution Timing, Pseudo-Records (:OLD, :NEW), and Evaluate Triggers vs Application-Coded Logic. 10 MARKS END-SEM Q1.e • Q5.b

DEFINITION

A **Database Trigger** is a named, stored PL/SQL block associated with a specific table or view that is **automatically executed (fired)** by the database engine whenever a specified event (such as `INSERT`, `UPDATE`, `DELETE`, or DDL/system event) occurs.

1. Trigger Classification & Components

- **Timing:**
  - `BEFORE`: Fires prior to executing the triggering statement. Used for validating data or modifying values before saving.
  - `AFTER`: Fires after executing the triggering statement. Used for auditing changes or cascading updates to other tables.
  - `INSTEAD OF`: Fires in place of the triggering statement. Primarily used on complex views that are otherwise not updatable.
- **Level:**
  - **Statement-Level Trigger:** Fires *exactly once* for the triggering SQL statement, regardless of how many rows are affected (even if 0 rows are affected). (Default level).
  - **Row-Level Trigger (`FOR EACH ROW`):** Fires *once for every single row* affected by the triggering statement. Required when inspecting individual row column values.
- **Pseudo-Records (`:OLD` and `:NEW`):**
  - In row-level triggers, `:OLD` refers to values before the change; `:NEW` refers to values after the change.
  - For `INSERT`: `:OLD` is null; `:NEW` contains the newly inserted values.
  - For `UPDATE`: `:OLD` contains values prior to update; `:NEW` contains updated values.
  - For `DELETE`: `:OLD` contains values being deleted; `:NEW` is null.

2. Evaluation: Database Triggers vs Manually Coded Application Logic

Evaluation Parameter	Database Triggers	Application-Coded Logic
Centralized Enforcement & Non-Bypassability	**Guaranteed.** Executes automatically on the database server. Even if a user connects via SQL Developer, Python script, or third-party tool, the trigger fires.	**Vulnerable to bypass.** Logic must be duplicated in every client app. Direct SQL access easily bypasses application checks.
Audit Trail Integrity	Reliable. Tamper-proof logging into audit tables occurs within the same atomic transaction.	Risky. If application crashes after writing to the main table but before audit logging, data is inconsistent.
Maintenance & Redundancy	Maintained in one central place. Changing business rules requires modifying only the database trigger.	Maintenance nightmare. Code is scattered across web frontends, mobile backends, and reporting tools.

Evaluation Parameter	Database Triggers	Application-Coded Logic
Debugging & Overhead	Cascading triggers can cause hidden performance penalties and mutating table errors (`ORA-04091`).	Easier to step through and debug within traditional IDEs.

Q26. Explain Stored Procedures and Stored Functions in PL/SQL. Syntax, Parameter Modes (IN, OUT, IN OUT), and Practical Comparison. 10 MARKS

CORE DEFINITIONS

Stored Procedure: A named, standalone PL/SQL program unit compiled and stored permanently in the database catalog that executes a sequence of operations (such as DML, transaction control, and auditing) on behalf of calling applications. It does not return a value in its header but can pass back multiple values via `OUT` or `IN OUT` parameters.

Stored Function: A named, standalone PL/SQL program unit compiled and stored in the database catalog that computes and **MUST return a single value** to the caller via an explicit `RETURN` statement.

1. Syntax Structure & Header Declarations

Stored Procedure Syntax

```
CREATE OR REPLACE PROCEDURE proc_name (
    param1 IN datatype,
    param2 OUT datatype,
    param3 IN OUT datatype
)
IS/AS
    -- Local variable declarations
BEGIN
    -- Executable statements
EXCEPTION
    -- Exception handling block
END proc_name;
/
```

Stored Function Syntax

```
CREATE OR REPLACE FUNCTION func_name (
    param1 IN datatype,
    param2 IN datatype
)
RETURN return_datatype
IS/AS
    -- Local variable declarations
BEGIN
    -- Executable statements
    RETURN computed_value;
EXCEPTION
    -- Exception handling block
END func_name;
/
```

2. The Three Formal Parameter Modes

Parameter Mode	Direction & Default Status	Characteristics & Modifiability	Valid Actual Arguments
IN	Input to subprogram (**Default** mode)	Acts as a read-only constant within the subprogram. Cannot be assigned to or modified.	Literals, constants, expressions, or initialized variables.
OUT	Output to caller	Used to return values back to the calling environment. Initialized to `NULL` upon entry; must be assigned a value.	Must be a variable (cannot pass literals, constants, or expressions).
IN OUT	Two-way (Input & Output)	Passes an initial value into the subprogram, allows modification during execution, and returns the modified value.	Must be a variable with an initial value.

3. Practical Implementation Examples

Practical Procedure: Bank Account Transfer

```
CREATE OR REPLACE PROCEDURE transfer_amount (
    p_from_acc IN NUMBER,
    p_to_acc   IN NUMBER,
    p_amount   IN NUMBER,
    p_status   OUT VARCHAR2
) IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccNo = p_from_acc;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccNo = p_to_acc;

    COMMIT;
    p_status := 'SUCCESS';
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        p_status := 'FAILED: ' || SQLERRM;
END transfer_amount;
/
```

Practical Function: Tax Calculation

```
CREATE OR REPLACE FUNCTION calc_income_tax (
    p_salary IN NUMBER
) RETURN NUMBER IS
    v_tax NUMBER(10,2);
BEGIN
    IF p_salary <= 250000 THEN
        v_tax := 0;
    ELSIF p_salary <= 500000 THEN
        v_tax := (p_salary - 250000) * 0.05;
    ELSE
        v_tax := 12500 + (p_salary - 500000) *
0.20;
    END IF;
    RETURN v_tax;
END calc_income_tax;
/
```

4. Invocation & Execution Differences

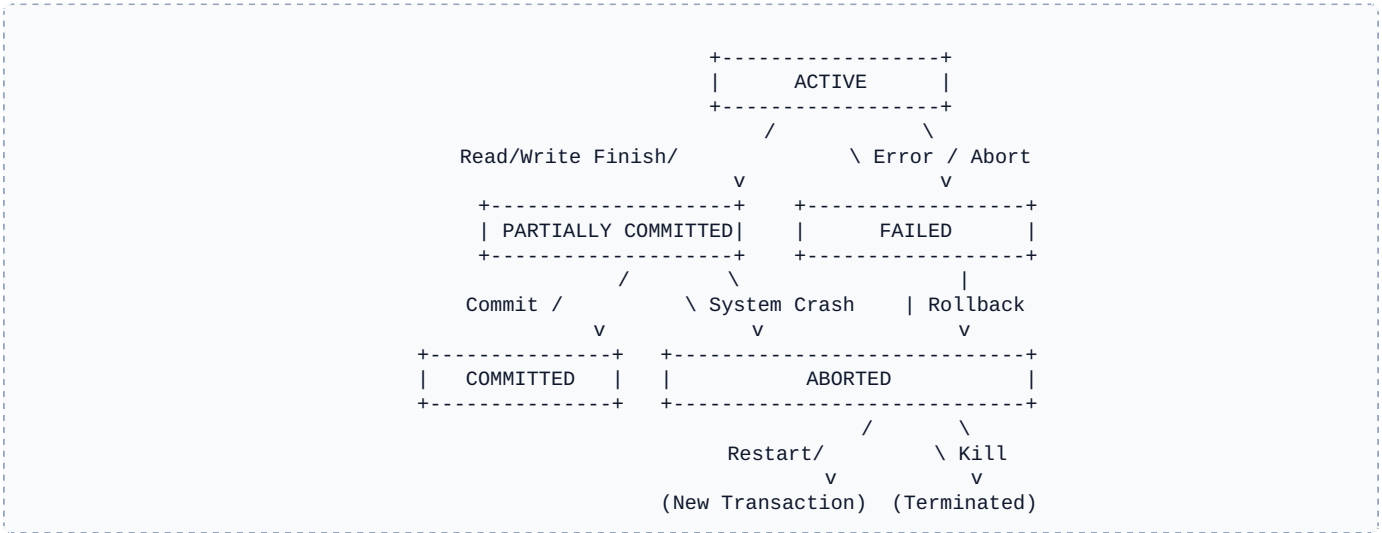
- **Invoking a Procedure:** Called as a standalone executable statement using the `EXECUTE` command in SQL*Plus / SQL Developer or within a PL/SQL block:
  `EXECUTE transfer_amount(101, 102, 5000, :v_res);`
  Restriction: Procedures **cannot** be called directly inside an SQL `SELECT`, `WHERE`, or `HAVING` clause!
- **Invoking a Function:** Can be called directly inside standard SQL queries, expressions, and views:
  `SELECT EmpName, Salary, calc_income_tax(Salary) AS TaxDue FROM Employee WHERE calc_income_tax(Salary) > 10000;`
  Or in PL/SQL: `v_final_tax := calc_income_tax(v_salary);`

Module 6: Transactions Management and Query Processing

Q27. Define a Transaction. Explain the Transaction State Transition Diagram with All States and Transitions. 5 MARKS

DEFINITION

A **Transaction** is a single logical unit of database processing that includes one or more database access operations (read, write, or update). It must execute completely or not at all (atomicity), transitioning the database from one consistent state to another consistent state.



Explanation of Transaction States

1. **Active:** The initial state. The transaction enters this state when it begins execution and stays here while executing read, write, and compute operations.
2. **Partially Committed:** Reached after the final statement has been executed, but before the commit changes are guaranteed to be safely written to permanent storage (e.g., commit records are still in volatile memory buffers).
3. **Failed:** Entered when normal execution cannot proceed due to hardware/software faults, internal arithmetic errors, or concurrency violations.
4. **Aborted:** The transaction has rolled back, undoing all partial modifications, and the database has been restored to the state it was in before the transaction started. The transaction can now be restarted or terminated.
5. **Committed:** Reached when the transaction successfully completes and all updates are safely written to non-volatile disk/log. Changes are permanent and cannot be undone.

Q28. Explain the ACID Properties of Transactions in Detail. Provide Practical Banking Examples and Explain How the DBMS Enforces Each. 10 MARKS END-SEM Q3.f • Q5.c

To preserve data integrity, the DBMS must guarantee the four **ACID** properties for all transactions:

1. Atomicity ("All or Nothing")

- **Concept:** A transaction is an indivisible unit of work; either all its operations are executed successfully and committed, or none of them are reflected in the database.
- **Banking Scenario:** Transfer ₹5,000 from Account A (balance ₹20,000) to Account B (balance ₹10,000).
  Steps: (1) `Read(A)`, (2) A = A - 5000, (3) `Write(A)`, (4) `Read(B)`, (5) B = B + 5000, (6) `Write(B)`.
  If a system crash occurs after step 3, Account A has been debited by ₹5,000 but Account B has not been credited. The DBMS must abort the transaction and restore Account A back to ₹20,000.
- **DBMS Enforcement:** Implemented by the **Recovery Manager** using **Write-Ahead Logging (WAL)** and undo-logging / shadow paging.

2. Consistency (Preservation of Correctness)

- **Concept:** Execution of a transaction in isolation must preserve the logical consistency of the database (satisfying all integrity constraints, balance rules, and invariants).

- **Banking Scenario:** The sum of balances (A + B) before the transfer is ₹30,000. After the transfer, (A + B) must strictly remain ₹30,000.
- **DBMS Enforcement:** Ensured partly by the programmer (writing correct logic) and enforced by the DBMS via **Integrity Constraints** (`CHECK` , `FOREIGN KEY` , triggers).

3. Isolation (Independence from Concurrent Transactions)

- **Concept:** Even if multiple transactions execute concurrently, each transaction must remain unaware of other concurrently executing transactions. Intermediate states must not be visible to others.
- **Banking Scenario:** If Transaction T_1 is executing the ₹5,000 transfer, a concurrent audit query T_2 calculating total bank assets must not read Account A after debit but Account B before credit!
- **DBMS Enforcement:** Implemented by the **Concurrency Control Manager** using locking protocols (e.g., Two-Phase Locking — 2PL) or timestamp ordering.

4. Durability (Permanence of Committed Data)

- **Concept:** Once a transaction successfully commits, its updates persist permanently in non-volatile storage, even if the database server crashes immediately afterwards.
- **Banking Scenario:** Once the customer receives an SMS confirming the ₹5,000 transfer, an immediate power failure must never erase the transfer.
- **DBMS Enforcement:** Implemented by the **Recovery Manager** by flushing transaction commit log records to non-volatile disk storage before acknowledging commit.

Q29. Discuss the Four Concurrency Control Anomalies in Multi-User Environments. 10 MARKS

Uncontrolled concurrent execution of transactions causes four major data inconsistency problems:

Anomaly	Failure Mechanism & Conflict Type	Concrete Example Scenario
1. Dirty Read (Temporary Update / Read-Write Conflict)	Occurs when Transaction T_2 reads an uncommitted intermediate value updated by Transaction T_1. If T_1 subsequently fails and aborts, T_2 has operated on phantom "dirty" data that never legally existed!	T_1 updates Account A from ₹10k to ₹5k. T_2 reads A = 5000 and calculates tax. T_1 then rolls back to ₹10k. T_2's calculation is invalid!
2. Lost Update (Write-Write Conflict)	Occurs when two transactions T_1 and T_2 read the same data item simultaneously, calculate new values independently, and overwrite each other's updates without including the intermediate modification.	Both T_1 and T_2 read flight seats S = 10. T_1 books 2 seats and writes S = 8. Concurrently, T_2 books 3 seats and writes S = 7. T_1's booking is completely lost!
3. Unrepeatable Read (Inconsistent Analysis / Read-Write Conflict)	Occurs when Transaction T_1 reads a data item twice. Between the two reads, Transaction T_2 updates the value and commits. T_1 receives two different values for the exact same read operation!	T_1 reads balance X = 5000. T_2 modifies X = 8000 and commits. T_1 reads X again and finds X = 8000, causing an inconsistent audit report.
4. Phantom Read	Occurs when Transaction T_1 executes a query using a search condition (e.g., `WHERE Dept = 'IT'`). Concurrently, T_2 inserts a brand new row satisfying that condition and commits. When T_1 repeats the query, a "phantom" row appears.	T_1 counts total IT employees (gets 10). T_2 inserts a new IT employee. T_1 counts again and gets 11 rows!

Q30. Explain Serializability. Compare Conflict Serializability and View Serializability. Detail the Precedence Graph Cycle Test. 10 MARKS END-SEM Q2.f • Q5.c

> **CORE CONCEPT**
> A non-serial schedule is **Serializable** if it is equivalent to some serial execution of the same transactions. Serializability is the fundamental criterion of correctness for concurrent schedules.

1. Conflicting Operations

Two operations I_i and I_j in a schedule conflict if and only if all three conditions are met:

1. They belong to different transactions ($i \neq j$).

2. They access the exact same data item *Q*.
3. **At least one of the operations is a write operation (`Write(Q)`).**
   Conflicting pairs: (1) $Read_i(Q)$ and $Write_j(Q)$, (2) $Write_i(Q)$ and $Read_j(Q)$, (3) $Write_i(Q)$ and $Write_j(Q)$.
   Non-conflicting pair: $Read_i(Q)$ and $Read_j(Q)$ (read-only operations never conflict).

2. Conflict Serializability & The Precedence Graph Algorithm

A schedule *S* is **Conflict Serializable** if it is conflict-equivalent to a serial schedule (it can be transformed into a serial schedule by swapping non-conflicting adjacent operations).

PRECEDENCE GRAPH (SERIALIZATION GRAPH) TESTING ALGORITHM

1. Construct a directed graph $G = (V, E)$ where vertices V represent all active transactions $\{T_1, T_2, \ldots, T_n\}$.
2. Draw a directed edge $T_i \rightarrow T_j$ if T_i executes an operation that conflicts with an operation executed later by T_j on the same data item *Q*.
3. **Cycle Detection Test:**
   • If the graph contains **NO CYCLES** (Acyclic) → The schedule is **Conflict Serializable**! The equivalent serial order is obtained via **Topological Sort**.
   • If the graph contains a **CYCLE** → The schedule is **NOT Conflict Serializable**!

3. View Serializability

A schedule *S* is **View Serializable** if it is view-equivalent to a serial schedule. Two schedules *S* and *S′* are view equivalent if:

- **Initial Read:** For each data item *Q*, if T_i reads the initial value of *Q* in *S*, then T_i must read the initial value of *Q* in *S′*.
- **Updated Read:** If T_i reads a value of *Q* produced by T_j in *S*, then T_i must also read the value produced by T_j in *S′*.
- **Final Write:** If T_i executes the final write on *Q* in *S*, then T_i must execute the final write on *Q* in *S′*.

KEY COMPARISON

Every conflict-serializable schedule is view-serializable, but the reverse is not true. Schedules containing **Blind Writes** (writing without reading first) can be view-serializable without being conflict-serializable. Testing view-serializability is **NP-Complete**, so DBMS engines implement conflict-serializability using lock protocols.

Q31. Explain Concurrency Control Using Lock-Based Protocols and the Two-Phase Locking (2PL) Protocol Variants. 10 MARKS END-SEM Q5.c

1. Lock Types & Compatibility

- **Shared Lock (S):** Read-only lock. Multiple transactions can hold shared locks concurrently on the same data item.
- **Exclusive Lock (X):** Read-and-Write lock. Only one transaction can hold an exclusive lock; all other lock requests are blocked.

Lock Compatibility	Shared (S)	Exclusive (X)
Shared (S)	Compatible (Granted)	Incompatible (Wait)
Exclusive (X)	Incompatible (Wait)	Incompatible (Wait)

2. Two-Phase Locking (2PL) Protocol

THE FUNDAMENTAL 2PL RULE

In Two-Phase Locking, all locking operations precede the first unlock operation. Each transaction must execute in two distinct phases:

1. **Growing Phase:** The transaction may obtain locks, but cannot release any lock.
2. **Shrinking Phase:** The transaction may release locks, but cannot acquire any new lock.

The transition point where the transaction acquires its final lock is called the **Lock Point**.

3. The Four Variants of 2PL

1. **Basic 2PL:** Follows standard growing and shrinking phases. Guarantees conflict serializability, but is susceptible to **Cascading Aborts** and **Deadlocks**.
2. **Strict 2PL:** Requires that all **Exclusive (X) locks** held by a transaction must be retained until the transaction commits or aborts.
   Advantage: Completely eliminates **Cascading Aborts** (ensures cascadeless recovery).
3. **Rigorous 2PL:** Requires that **ALL locks (both Shared and Exclusive)** must be held until the transaction commits or aborts. Most commonly implemented in commercial systems.
4. **Conservative (Static) 2PL:** Requires a transaction to lock all required data items simultaneously before execution begins. If any item is unavailable, it acquires none and waits.
   Advantage: Completely **Deadlock-Free**, but severely hampers concurrency.

Q32. Explain Database Recovery Management: Log-Based Recovery (WAL, Checkpoints) and Shadow Paging. 10 MARKS END-SEM Q5.c

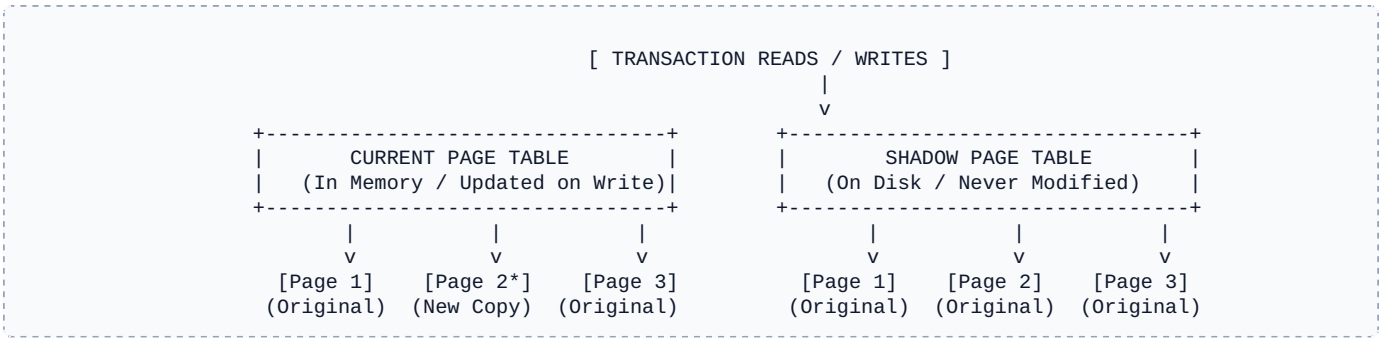
1. Log-Based Recovery & Write-Ahead Logging (WAL)

The **Log** is an append-only, sequential record of all update activities stored on stable storage. Each log record has the format: `<Ti, X, Old_Value, New_Value>`.

> **WRITE-AHEAD LOGGING (WAL) RULE**
> Before any database modification is written to disk blocks, the corresponding log record must be flushed to stable storage. This guarantees that if a crash occurs, the recovery manager can inspect the log and reconstruct the correct state.

- **Deferred Database Modification:** Database updates are postponed until after the transaction enters the partially committed state.
  Recovery action: Only `REDO` is needed; no `UNDO` is ever required!
- **Immediate Database Modification:** Updates can be written to disk while the transaction is still active.
  Recovery action: Requires both `UNDO` (for active/failed transactions) and `REDO` (for committed transactions).
- **Checkpoints:** Periodically, the DBMS flushes all dirty memory buffers to disk and writes a `<CHECKPOINT L>` record to the log. During recovery, the engine only needs to scan back to the most recent checkpoint, drastically reducing recovery time.

2. Shadow Paging Recovery System



How Shadow Paging Works

- The database maintains two page tables throughout a transaction's lifetime:
  - **Shadow Page Table:** Points to the original, committed pages on disk. It is **never modified** during transaction execution.
  - **Current Page Table:** Points to database pages during execution. When an active transaction writes to a page, a new block is allocated on disk, the content is copied and modified, and the pointer in the Current Page Table is updated to the new block.
- **Commit Logic:** When the transaction commits, all modified blocks are written to disk, and the pointer to the Shadow Page Table is atomically switched to point to the Current Page Table. The old pages are returned to the free list.
- **Crash Recovery:** Instantaneous! If a crash occurs before commit, the DBMS simply discards the Current Page Table. The Shadow Page Table on disk still references the consistent database state prior to transaction start. **Zero UNDO and zero REDO operations are required!**

- **Disadvantages:** High disk fragmentation and overhead of copying large page tables.

Q33. Provide an Overview of Query Processing and Heuristic Query Optimization. 5 MARKS

1. Steps in Query Processing

1. **Parsing and Translation:** Translates incoming SQL text into internal parse trees, checks syntax, verifies table/column names against the system catalog, and translates the query into an initial relational algebra expression.
2. **Optimization:** Explores alternative, logically equivalent evaluation plans and chooses the execution plan with the lowest estimated cost (disk I/O and CPU time).
3. **Code Generation / Evaluation Engine:** Executes the selected query evaluation plan against database files and returns the resulting tuple stream to the client.

2. Heuristic Query Optimization Rules

Heuristic optimization transforms relational algebra trees using equivalence rules to minimize intermediate relation sizes:

- **Perform Selection (σ) as Early as Possible (Push Down Selections):** Filtering rows immediately drastically reduces the number of tuples passed to expensive Cartesian products and joins.
- **Perform Projection (π) as Early as Possible (Push Down Projections):** Pruning unneeded attributes early reduces the tuple width and memory footprint of intermediate buffers.
- **Perform Most Restrictive Selection / Join First:** When joining multiple relations, execute joins that produce the smallest intermediate relation sizes first.

CATEGORY 2: MASTER COMPARISON TABLES ("DIFFERENTIATE BETWEEN")

Complete collection of all syllabus comparisons. Formatted in structured, multi-parameter academic tables specifically optimized for 4, 5, and 10 mark exam scoring.

Table 1: File Processing System vs Database Management System (DBMS) 5 MARKS END-SEM Q1.a • CA1 Q1

Parameter / Basis	File Processing System	Database Management System (DBMS)
Data Redundancy & Inconsistency	High. The same data is duplicated across multiple independent application files, leading to inconsistency when updated.	Minimal. Centralized repository eliminates redundancy; any single update reflects across all views automatically.
Data Sharing	Extremely difficult. Data is isolated in distinct file formats; sharing requires writing custom conversion programs.	Seamless. Multi-user concurrent access is natively supported with fine-grained authorization.
Data Independence	Absent. File structures and record formats are hardcoded directly into application source code (Program-Data Dependence).	Guaranteed. Schema is stored in the system catalog, providing both physical and logical data independence.
Concurrent Access & Anomalies	Uncontrolled. Simultaneous access easily results in lost updates, race conditions, and corrupted records.	Robust. Concurrency control protocols (e.g., 2PL) ensure serializability and eliminate race conditions.
Security & Access Control	Primitive. Relies solely on OS file permissions (read/write/execute); cannot restrict access to specific rows or columns.	Advanced. Granular privileges (DCL `GRANT` / `REVOKE`), view-based masking, and role-based access control.
Integrity Constraints	Must be manually enforced inside every application program; difficult to maintain and enforce consistently.	Enforced declaratively at the engine level (`PRIMARY KEY` , `FOREIGN KEY` , `CHECK`).
Crash Recovery	Lacks automated rollback. If a crash occurs mid-write, data is corrupted or left half-updated.	Guaranteed Atomicity and Durability via Write-Ahead Logging (WAL) and recovery algorithms.

Table 2: Physical Data Independence vs Logical Data Independence 5 MARKS END-SEM Q3.a • CA1 Q7

Parameter	Physical Data Independence	Logical Data Independence
Definition	The capacity to modify the internal (physical) schema without having to alter the conceptual or external schemas.	The capacity to modify the conceptual schema without having to alter external schemas or application programs.
Level of Architecture	Operates between the **Internal Level** and the **Conceptual Level**.	Operates between the **Conceptual Level** and the **External (View) Level**.
Mapping Responsible	Adjusted via **Conceptual-Internal Mapping**.	Adjusted via **External-Conceptual Mapping**.
Ease of Realization	**Relatively Easy** to achieve; changes purely affect storage and indexing performance.	**Difficult** to achieve; applications are heavily dependent on the logical attributes and types of entities.
Typical Examples	Switching from sequential file to B+ Tree index; repartitioning tables; changing disk storage devices or block sizes.	Adding a new column (e.g., `AadhaarNo`); adding a new entity/relation; expanding constraints without breaking views.

Table 3: Strong Entity Set vs Weak Entity Set 5 MARKS

Parameter	Strong Entity Set	Weak Entity Set
Primary Key	Possesses an explicit **Primary Key** formed purely from its own attributes.	Does NOT possess sufficient attributes to form a primary key on its own. Has only a **Partial Key (Discriminator)**.
Existence Dependency	Independent. Entities can exist without depending on the existence of any other entity.	Dependent. Entities cannot exist in the database without an associated **Owner (Identifying) Entity**.
Primary Key Formation	Formed directly from its own candidate key (e.g., `EmpId`).	Composite key formed by: `{Primary Key of Owner Entity + Partial Key of Weak Entity}`.
ER Diagram Symbol	Drawn as a single **Rectangle**; primary key has a **Solid Underline**.	Drawn as a **Double Rectangle**; partial key has a **Dashed Underline**.
Relationship Symbol	Connects via standard single **Diamond**.	Connects to owner entity via an **Identifying Relationship (Double Diamond)** with total participation (double line).
Real-World Example	`EMPLOYEE(EmpId, Name, Salary)`	`DEPENDENT(DepName, Relationship, DOB)` dependent on `EMPLOYEE`.

Table 4: Generalization vs Specialization vs Aggregation 5 MARKS END-SEM Q2.a • CA1 Q6

Parameter	Generalization	Specialization	Aggregation
Design Approach	**Bottom-Up** design process.	**Top-Down** design process.	**Relationship Abstraction** process.
Core Concept	Extracts shared attributes from multiple low-level entities to synthesize one generalized high-level entity.	Splits a high-level entity into specialized sub-entities having distinct attributes or relationships.	Treats a relationship and its participating entities as a higher-level composite entity.
Attribute Inheritance	Higher-level entity holds common attributes; lower-level entities inherit them.	Subclasses inherit all attributes and relationships of the superclass plus their own unique ones.	Not an inheritance hierarchy; allows an entire relationship to participate in another relationship.
ER Notation	Triangle labeled `IS-A` pointing upwards toward generalized entity.	Triangle labeled `IS-A` pointing downwards toward specialized sub-entities.	A bounding rectangle enclosing the relationship diamond and participating entities.
Example	`CAR` and `TRUCK` generalized into `VEHICLE`.	`EMPLOYEE` specialized into `SALARIED_EMP` and `HOURLY_EMP`.	Relationship `WORKS_ON(Emp, Project)` aggregated to link with `MACHINERY`.

Table 5: Super Key vs Candidate Key vs Primary Key vs Alternate Key 5 MARKS CA1 Q9

Key Type	Definition & Uniqueness Rule	Minimality Property	Nullability Rule
Super Key (SK)	Any set of one or more attributes that uniquely identifies a tuple in a relation.	**Not minimal** (May include redundant attributes, e.g., `{RollNo, Name}`).	Can theoretically contain nulls if non-null subpart ensures uniqueness.
Candidate Key (CK)	A minimal super key from which no attribute can be discarded without destroying uniqueness.	**Strictly Minimal** (No proper subset is a super key).	Attributes may be null only if not chosen as primary key.

Key Type	Definition & Uniqueness Rule	Minimality Property	Nullability Rule
Primary Key (PK)	The single candidate key officially designated by the database designer to identify tuples uniquely.	**Strictly Minimal**.	**Strictly NOT NULL** (Entity Integrity constraint). Exactly 1 per table.
Alternate Key (AK)	All remaining candidate keys that were not selected as the primary key.	**Strictly Minimal**.	May allow `NULL` depending on business rules (enforced via `UNIQUE`).

Table 6: Primary Key vs Unique Key vs Foreign Key 5 MARKS

Parameter	Primary Key	Unique Key	Foreign Key
Purpose	Uniquely identifies each record in the relation.	Prevents duplicate values in specific column(s).	Enforces referential integrity with another table.
NULL Values Allowed	**NEVER** allows `NULL` values.	Allows `NULL` values (multiple nulls permitted).	Allows `NULL` (unless explicitly declared `NOT NULL`).
Quantity Allowed	**Exactly ONE** per table.	**Multiple** unique keys allowed per table.	**Multiple** foreign keys allowed per table.
Default Indexing	Automatically creates a unique clustered index (in most engines).	Automatically creates a unique non-clustered index.	Does not create an index automatically (recommended to index manually).
Cross-Table Link	Referenced by foreign keys in child tables.	Can also be referenced by foreign keys.	References a Primary or Unique Key in a parent table.

Table 7: Unary Operations vs Binary Operations in Relational Algebra 5 MARKS CA1 Q14

Parameter	Unary Operations	Binary Operations
Number of Operands	Operates on **exactly one** input relation.	Operates on **two** input relations.
Primary Operators	Selection (σ), Projection (π), Rename (ρ).	Union (∪), Intersection (∩), Difference (−), Cartesian Product (×), Join (⋈), Division (÷).
Degree of Output	Equal to input (for σ, ρ) or ≤ input (for π).	Sum of input degrees (for ×, ⋈) or same degree (for set operations ∪, ∩, −).
Union Compatibility	Not applicable.	Strictly mandatory for Set Operations (∪, ∩, −); not required for × and ⋈.

Table 8: Selection (σ) vs Projection (π) in Relational Algebra 5 MARKS END-SEM Q3.b

Parameter	Selection (σ)	Projection (π)
Filtering Dimension	**Horizontal partitioning** (filters rows/tuples).	**Vertical partitioning** (filters columns/attributes).
Mathematical Symbol	Greek letter Sigma ($\sigma_{predicate}(R)$).	Greek letter Pi ($\pi_{attribute_list}(R)$).
Condition / Parameter	Requires a Boolean propositional condition.	Requires a comma-separated list of desired attribute names.
Impact on Degree	**Preserves degree**: Output relation has the exact same number of columns as input relation.	**Reduces or maintains degree**: Output degree equals number of specified attributes.
Impact on Cardinality	Output cardinality ≤ input relation cardinality.	Output cardinality ≤ input (duplicate rows are eliminated by relational algebra set semantics).
SQL Equivalent	The `WHERE` clause.	The `SELECT [column_list]` clause.

Parameter	Selection (σ)	Projection (π)
Example	$\sigma_{Salary > 50000}(Employee)$	$\pi_{Name, Salary}(Employee)$

Table 9: Theta Join vs Equijoin vs Natural Join 5 MARKS

Parameter	Theta Join ($R \bowtie_{\theta} S$)	Equijoin ($R \bowtie_{R.A = S.B} S$)	Natural Join ($R \bowtie S$)
Condition Operator	Any comparison operator: =, <, ≤, >, ≥, ≠.	Strictly the equality operator (=).	Implicit equality on all identically named common attributes.
Common Attribute Handling	Retains all columns from both tables; common attributes appear twice.	Retains all columns from both tables; join columns appear twice.	**Eliminates duplicate columns automatically**; common attributes appear exactly once!
Degree of Result	Degree(R) + Degree(S)	Degree(R) + Degree(S)	Degree(R) + Degree(S) − (Number of Common Attributes)
SQL Syntax	`FROM R JOIN S ON R.A > S.B`	`FROM R JOIN S ON R.A = S.A`	`FROM R NATURAL JOIN S`

Table 10: Inner Join vs Left Join vs Right Join vs Full Outer Join 5 MARKS CA2 Sec A Q6

Join Type	Tuples Included in Result	Padding for Unmatched Tuples
INNER JOIN	Only tuples that satisfy the join condition in **both tables** (intersection of keys).	None (Unmatched tuples from both relations are discarded).
LEFT OUTER JOIN	**All tuples from the Left table**, plus matching tuples from the Right table.	Right table attributes are padded with `NULL` if no match exists.
RIGHT OUTER JOIN	**All tuples from the Right table**, plus matching tuples from the Left table.	Left table attributes are padded with `NULL` if no match exists.
FULL OUTER JOIN	**All tuples from both Left and Right tables**, regardless of whether a match exists.	Attributes from either table are padded with `NULL` whenever an unmatched tuple is present.

Table 11: DELETE vs TRUNCATE vs DROP in SQL 5 MARKS END-SEM Q3.c

Parameter	DELETE	TRUNCATE	DROP
Command Category	**DML** (Data Manipulation Language).	**DDL** (Data Definition Language).	**DDL** (Data Definition Language).
Target & Effect	Deletes specified rows (or all rows if no WHERE clause). Table structure remains intact.	Deletes **ALL rows** at once. Table structure and schema remain intact.	Completely destroys the **entire table structure, schema, constraints, and data**.
WHERE Clause	**Supported.** (Can selectively delete specific rows).	**Not Supported.** (All rows are deleted unconditionally).	**Not Supported.**
Rollback / Transaction	**Can be Rolled Back.** (Logged row-by-row in the transaction log).	**Cannot be Rolled Back** in standard practice (Auto-committed DDL; deallocates pages).	**Cannot be Rolled Back.** (Auto-committed; drops catalog entries).
Storage Space Deallocation	Does not deallocate freed storage space back to the tablespace.	**Deallocates disk pages** immediately, resetting high-water mark.	Completely releases all table storage space back to the operating system/tablespace.
Triggers	Fires `ON DELETE` database triggers for affected rows.	**Does NOT fire** delete triggers.	Does not fire DML triggers (drops trigger definitions).

Parameter	DELETE	TRUNCATE	DROP
Speed / Performance	Slowest (Generates extensive undo/redo log records for each individual row).	Much faster than DELETE (Deallocates entire data blocks at page level).	Instantaneous.

Table 12: WHERE Clause vs HAVING Clause in SQL 5 MARKS

Parameter	WHERE Clause	HAVING Clause
Filtering Target	Filters **individual rows** before any grouping occurs.	Filters **aggregated groups** after `GROUP BY` has been performed.
Aggregate Functions	**CANNOT** contain aggregate functions (e.g., `WHERE AVG(Salary) > 50000` is illegal).	**CAN** contain aggregate functions (e.g., `HAVING AVG(Salary) > 50000`).
Query Placement	Placed before the `GROUP BY` clause.	Placed after the `GROUP BY` clause.
Requirement of GROUP BY	Can be used independently without a `GROUP BY` clause.	Typically requires a `GROUP BY` clause (or filters over the entire table as a single group).
Execution Stage	Applied directly during table scanning / index scanning phase.	Applied in post-processing after group partitions are aggregated.

Table 13: Subquery Types: Single-Row vs Multi-Row vs Correlated Subquery 5 MARKS

Parameter	Single-Row Subquery	Multi-Row Subquery	Correlated Subquery
Cardinality of Output	Returns exactly **one row and one column** (scalar).	Returns **multiple rows** (a list or column of values).	Can return single or multiple rows.
Applicable Operators	Scalar operators: `=, <, >, <=, >=, <>`	List operators: `IN, NOT IN, ANY, ALL`	Existence operators: `EXISTS, NOT EXISTS` or scalar ops.
Execution Frequency	Executes **once** before outer query runs.	Executes **once** before outer query runs.	Executes **once for every candidate row** evaluated by the outer query.
Dependency on Outer Query	**Independent.** Inner query has no references to outer query columns.	**Independent.** Inner query can be run standalone.	**Dependent.** Inner query explicitly references an outer table alias.
Performance	Very fast.	Fast (can leverage set matching).	Can be slower on large tables (O(N × M) unless indexed or unnested by optimizer).

Table 14: Base Table vs View in SQL 5 MARKS

Parameter	Base Table	View
Physical Data Storage	Stores actual data records physically on non-volatile disk.	Does NOT store data (virtual table); only the SQL definition is saved in the data dictionary.
Storage Space Requirement	Requires disk space proportional to the number of records and columns.	Consumes negligible disk space (only metadata text is stored).
Performance	Fast direct access via clustered/non-clustered indices.	Slight overhead as query definition must be merged and executed against base tables.
DML Updates	Fully supports all `INSERT`, `UPDATE`, `DELETE` operations unconditionally.	Restricted. Updatable only if derived from a single table without aggregations, `DISTINCT`, or `GROUP BY`.
Security Role	Exposes full schema unless restricted by permissions.	Primary tool for row-level and column-level security and data abstraction.

Table 15: Partial Dependency vs Transitive Dependency vs Full Functional Dependency 5 MARKS

CA2 Sec B Q4

Dependency Type	Formal Definition	Normal Form Violated	Concrete Example
Full Functional Dependency	An FD $X \rightarrow Y$ is full if removal of any attribute A from X causes the dependency to no longer hold ($X - \{A\} \not\rightarrow Y$).	**None** (Desirable target condition for 2NF relations).	In *R(RollNo, CourseId, Marks)*, *{RollNo, CourseId} → Marks* is full.
Partial Dependency	An FD $X \rightarrow Y$ is partial if some attribute $A \in X$ can be removed and the dependency still holds ($X - \{A\} \rightarrow Y$, where X is a candidate key).	**Violates 2NF!** (Must be eliminated to attain 2NF).	In *R(RollNo, CourseId, StudentName)*, *RollNo → StudentName* is partial.
Transitive Dependency	An FD $X \rightarrow Z$ is transitive if there exists an attribute set Y (neither a candidate key nor a subset of any key) such that $X \rightarrow Y$ and $Y \rightarrow Z$ hold.	**Violates 3NF!** (Must be eliminated to attain 3NF).	In *R(EmpId, DeptId, DeptName)*, *EmpId → DeptId* and *DeptId → DeptName*.

Table 16: Comprehensive Comparison of Normal Forms: 1NF vs 2NF vs 3NF vs BCNF

10 MARKS

END-SEM Q5.a

Normal Form	Core Condition / Requirement	Undesirable Dependency Eliminated	Formal Condition on $X \rightarrow Y$
1NF	All attribute values must be atomic. No multivalued attributes or repeating groups.	Non-atomic values, composite attributes, and nested arrays.	Attributes contain scalar values only.
2NF	Must be in 1NF. Every non-prime attribute must be fully functionally dependent on every candidate key.	**Partial Dependencies** (dependency on a proper subset of a composite candidate key).	No non-prime attribute A depends on $X \subset CK$.
3NF	Must be in 2NF. No non-prime attribute is transitively dependent on any candidate key.	**Transitive Dependencies** between non-prime attributes.	For every non-trivial FD $X \rightarrow Y$, either: 1. X is a **Super Key**, OR 2. Y is a **Prime Attribute**.
BCNF	Strict version of 3NF. In every non-trivial functional dependency, the left-hand determinant must be a super key.	Anomalies resulting from overlapping composite candidate keys.	For every non-trivial FD $X \rightarrow Y$: ***X* MUST be a Super Key!** (No exceptions for prime attributes).

Table 17: Lossless-Join Decomposition vs Lossy Decomposition

5 MARKS

END-SEM Q1.d • CA2 Sec B Q5

Parameter	Lossless-Join Decomposition	Lossy Decomposition
Definition	A decomposition where natural join of decomposed relations exactly reconstructs the original relation: $R_1 \bowtie R_2 = R$.	A decomposition where natural join generates **spurious (fake/ghost) tuples**: $R_1 \bowtie R_2 \supset R$.
Mathematical Test	At least one common attribute dependency must hold in F^+: $(R_1 \cap R_2) \rightarrow R_1$ OR $(R_1 \cap R_2) \rightarrow R_2$.	Neither holds: $(R_1 \cap R_2) \nrightarrow R_1$ and $(R_1 \cap R_2) \nrightarrow R_2$. Common attributes do NOT form a key.
Information Preservation	**100% Preserved.** Zero information loss and zero false data generated.	**Information is Lost.** Even though more rows are generated, the ability to distinguish valid records is lost.
Suitability	**Mandatory** requirement for all database normalization schemes.	**Unacceptable** in production database design.

Table 18: Dependency Preserving vs Non-Preserving Decomposition

5 MARKS

CA2 Sec B Q6

Parameter	Dependency Preserving Decomposition	Non-Preserving Decomposition
Definition	All original functional dependencies can be enforced within individual decomposed tables without joining: $(F_1 \cup F_2 \ldots \cup F_n)^+ = F^+$.	At least one functional dependency in F spans multiple decomposed relations and cannot be checked locally.
Constraint Validation Cost	**Low cost.** Validations occur during single-table inserts/updates using local primary/unique keys.	**Extremely high cost.** Enforcing cross-table dependencies requires an expensive multi-table join on every insert.
Attainability in 3NF & BCNF	Always achievable in **3NF**.	Sometimes unavoidable when normalizing to **BCNF** (BCNF may sacrifice dependency preservation).

Table 19: SQL vs PL/SQL 5 MARKS END-SEM Q1.e

Parameter	SQL (Structured Query Language)	PL/SQL (Procedural Language/SQL)
Language Nature	**Declarative** (specifies *what* data to retrieve, not how to retrieve it).	**Procedural** (specifies *how* tasks are performed step-by-step using algorithms).
Flow Control & Variables	No variables, conditional branching (`IF`), or looping (`LOOP`) constructs.	Full support for variables, conditional branches (`IF-THEN-ELSE` , `CASE`), and loops (`WHILE` , `FOR`).
Execution Unit	Executes a **single statement at a time**.	Executes an entire **block of statements** together.
Network Traffic	High network traffic (each query requires a separate round-trip to the server).	Significantly reduced network traffic (entire block sent in one network trip).
Error Handling	No structured runtime exception handling.	Dedicated `EXCEPTION` block for handling runtime errors.

Table 20: Stored Procedure vs Stored Function in PL/SQL 5 MARKS

Parameter	Stored Procedure	Stored Function
Return Value	**Does NOT mandate a return value.** (Can return 0, 1, or multiple values via `OUT` parameters).	**MUST return exactly one value** via the mandatory `RETURN` clause.
Calling in SQL Statements	**Cannot** be called directly inside SQL `SELECT` , `WHERE` , or `HAVING` clauses.	**Can be invoked directly** inside SQL statements (e.g., `SELECT Name, CalcTax(Salary) FROM Emp`).
DML Operations	Freely executes `INSERT` , `UPDATE` , `DELETE` operations.	When called from a `SELECT` statement, it **cannot perform DML** on tables (purity rule).
Parameter Modes	Supports `IN` , `OUT` , and `IN OUT` modes.	Typically uses only `IN` parameters (discouraged from using `OUT`).
Primary Purpose	Executing business logic, workflows, batch processing, and transactional updates.	Computing and returning a calculated result or formula.

Table 21: Implicit Cursor vs Explicit Cursor in PL/SQL 5 MARKS

Parameter	Implicit Cursor	Explicit Cursor
Creation & Management	Created and managed **automatically by Oracle** engine for all single-row DML and `SELECT INTO` statements.	Declared, opened, fetched, and closed **explicitly by the programmer**.
Number of Rows Handled	Intended for queries returning **exactly one row**. (Throws `TOO_MANY_ROWS` if query returns > 1 row).	Designed specifically to handle queries returning **multiple rows**.
Control & Lifecycle	Programmer has no control over lifecycle; opened and closed internally.	Full programmatic control over `OPEN` , `FETCH` , and `CLOSE` operations.
Attribute Access	Prefixed by `SQL%` (e.g., `SQL%FOUND` , `SQL%ROWCOUNT`).	Prefixed by the declared cursor name (e.g., `cur_emp%FOUND`).

Table 22: Row-Level Trigger vs Statement-Level Trigger in PL/SQL 5 MARKS

Parameter	Row-Level Trigger	Statement-Level Trigger
Declaration Clause	Includes the **`FOR EACH ROW`** clause in trigger definition.	Does NOT include `FOR EACH ROW` clause (Default trigger behavior).

Parameter	Row-Level Trigger	Statement-Level Trigger
Execution Frequency	Fires **once for every single row** affected by the triggering statement (if 100 rows updated, fires 100 times).	Fires **exactly once** for the statement as a whole, regardless of how many rows are affected (even if 0 rows).
Access to Old/New Values	**Can access** `:OLD` and `:NEW` column values for each row.	**Cannot access** `:OLD` and `:NEW` values.
Typical Use Cases	Data validation, auditing row changes, populating auto-generated primary keys.	Restricting access during off-hours, logging general table access, checking batch constraints.

Table 23: Database Triggers vs Stored Procedures 5 MARKS END-SEM Q1.e

Parameter	Database Trigger	Stored Procedure
Invocation Mechanism	**Implicit / Automatic.** Fired by the database engine in response to DML/DDL events.	**Explicit.** Invoked manually by a user or application call (`EXEC procedure_name`).
Parameters	**Cannot accept parameters.** (No `IN` , `OUT` parameters allowed).	**Accepts parameters** freely (supports `IN` , `OUT` , `IN OUT` modes).
Transaction Control	Cannot issue `COMMIT` , `ROLLBACK` , or `SAVEPOINT` (runs within caller transaction).	Can freely execute transaction control statements (`COMMIT` , `ROLLBACK`).
Primary Role	Automated auditing, enforcing complex integrity rules, security logging.	Encapsulating business calculations, batch updates, and reusable logic.

Table 24: Serial Schedule vs Serializable Schedule 5 MARKS

Parameter	Serial Schedule	Serializable Schedule
Transaction Interleaving	**No interleaving.** One transaction executes from start to finish before the next begins.	**Interleaved execution.** Operations from multiple transactions are interleaved concurrently.
Execution Speed & Throughput	Poor throughput and high response times (CPU sits idle during disk I/O of active transaction).	High throughput and reduced response times (CPU switches between transactions during disk I/O).
Consistency Guarantee	Guaranteed consistent by definition (assuming individual transactions are correct).	Guaranteed consistent because its final effect is mathematically equivalent to a serial schedule.

Table 25: Conflict Serializability vs View Serializability 5 MARKS END-SEM Q5.c

Parameter	Conflict Serializability	View Serializability
Equivalence Basis	Based on the relative ordering of **conflicting operations** (R-W, W-R, W-W).	Based on **initial read, updated read, and final write** invariants.
Testing Mechanism	Tested in polynomial time ($O(V + E)$) using an **Acyclic Precedence Graph**.	Testing is **NP-Complete** (computationally intractable for large schedules).
Blind Writes Handling	Cannot handle blind writes effectively (often falsely rejects them).	Correctly identifies schedules with **Blind Writes** as serializable.
Subset Relationship	Conflict Serializable ⊆ View Serializable. (All conflict serializable schedules are view serializable).	Broader class. View Serializable includes schedules that are not conflict serializable.
Practical Implementation	Directly implemented in commercial DBMS engines using Two-Phase Locking (2PL).	Purely theoretical; not implemented directly in DBMS engines due to NP-completeness.

Table 26: Shared Lock (S) vs Exclusive Lock (X) 5 MARKS

Parameter	Shared Lock (S)	Exclusive Lock (X)
Permitted Operations	**Read-Only.** The transaction holding the lock can only read data item *Q*.	**Read and Write.** The transaction can both read and write data item *Q*.
Concurrent Holders	**Multiple transactions** can hold Shared locks concurrently on the same item.	**Only ONE transaction** can hold an Exclusive lock on the item at any given time.
Compatibility Matrix	Compatible with other Shared locks; incompatible with Exclusive locks.	Incompatible with ALL other locks (both Shared and Exclusive).
SQL Command Locking	Acquired by `SELECT ... LOCK IN SHARE MODE`.	Acquired by `SELECT ... FOR UPDATE`, `INSERT`, `UPDATE`, `DELETE`.

Table 27: Comparison of Two-Phase Locking (2PL) Variants 10 MARKS END-SEM Q5.c

2PL Variant	Lock Acquisition Rule	Lock Release Rule	Key Guarantees & Drawbacks
Basic 2PL	Locks acquired dynamically during growing phase.	Locks can be released anytime during shrinking phase.	Guarantees Conflict Serializability. **Prone to Cascading Aborts & Deadlocks.**
Strict 2PL	Locks acquired dynamically during growing phase.	**Exclusive (X) locks held until Commit / Abort.** Shared locks released during shrinking phase.	**Eliminates Cascading Aborts** (Cascadeless). Still susceptible to Deadlocks.
Rigorous 2PL	Locks acquired dynamically during growing phase.	**ALL locks (S and X) held until Commit / Abort.** Shrinking phase happens at end of transaction.	Cascadeless and simple to implement. Most widely used in commercial DBMS. Deadlocks still possible.
Conservative 2PL (Static)	**All required locks acquired simultaneously** before transaction execution begins.	Locks released during shrinking phase.	**Completely Deadlock-Free!** Low concurrency; hard to predict all needed locks upfront.

Table 28: Log-Based Recovery vs Shadow Paging 10 MARKS END-SEM Q5.c

Parameter	Log-Based Recovery (WAL)	Shadow Paging Recovery
Storage Mechanism	Maintains an append-only sequential log file on stable storage with transaction operation records.	Maintains two page tables: a **Shadow Page Table** (on disk) and a **Current Page Table** (in memory).
Update Mechanism	Updates are applied in place on database blocks in buffer cache, guided by log records.	Copy-on-write: Modifies a newly allocated disk page while leaving the original page untouched.
Crash Recovery Speed	Slower. Recovery requires scanning log records, executing `UNDO` and `REDO` lists.	**Instantaneous!** Simply discards the Current Page Table pointer. **No UNDO, No REDO!**
Data Fragmentation	Low. Blocks remain organized contiguously in physical database files.	**High Fragmentation.** Copy-on-write scatters pages across non-contiguous disk sectors.
Commit Overhead	Flushing the log buffer to disk.	Flushing modified pages, updating page table pointer, freeing old blocks.

Table 29: Deferred Database Modification vs Immediate Database Modification 5 MARKS END-SEM Q5.c

Parameter / Basis	Deferred Database Modification (NO-UNDO / REDO)	Immediate Database Modification (UNDO / REDO)
Timing of Disk Updates	Database disk blocks are updated **only after** the transaction commits (reaches partially committed state).	Database disk blocks can be updated **while the transaction is still active**, prior to commit.
Log Record Format	Only requires the new value: `<T_i, X, New_Value>`. (Old value not needed since disk is not modified early).	Must record both before and after values: `<T_i, X, Old_Value, New_Value>`.
Crash Recovery Actions	**REDO only!** Active/uncommitted transactions never touched disk, so **NO UNDO** is needed.	**Both UNDO and REDO!** Active transactions that wrote dirty blocks to disk must be `UNDO`ed.
Buffer Cache Overhead	High memory footprint: all updates must be buffered in RAM until commit.	Low memory footprint: buffer manager can flush dirty blocks to disk anytime under memory pressure.
Write-Ahead Logging (WAL)	Commit record `<T_i COMMIT>` must be flushed to stable log before dirty buffer pages are written.	Log record must be flushed to disk **before** the corresponding data block is written to disk.
Transaction Rollback Cost	Instantaneous: Simply discard uncommitted memory buffer blocks and ignore log records.	Expensive: Recovery manager must traverse log backwards and execute compensating write operations (UNDO).
Throughput & Suitability	Ideal for short, read-heavy, low-concurrency transactions. Unsuitable for huge batch updates.	Industry standard for high-performance, long-running OLTP database engines.

CATEGORY 3: CODE, QUERIES & PRACTICAL NUMERICALS

Step-by-step solved practical problems, complete working code implementations, relational algebra expressions, normalization numericals, and transaction precedence graphs.

3.1 Relational Algebra Solved Queries

Problem 1: Employee Salary Filtering 4 MARKS END-SEM Q1.b

Question: Write a relational algebra expression to select all tuples from a relation `EMPLOYEE` where the salary is greater than 50,000.

$$\sigma_{Salary > 50000}(EMPLOYEE)$$

Explanation: The selection operator (σ) filters rows horizontally based on the predicate `Salary > 50000`. All attributes/columns of matching tuples are retained in the result relation.

Problem 2: Student Enrollment Names 5 MARKS END-SEM Q2.b

Question: Given two relations:
`STUDENT(SID, SNAME, DEPT)`
`ENROLL(SID, COURSEID)`
Write a relational algebra query to find the names of students enrolled in any course.

$$\pi_{SNAME}(STUDENT \bowtie_{STUDENT.SID = ENROLL.SID} ENROLL)$$

Or using Natural Join:

$$\pi_{SNAME}(STUDENT \bowtie ENROLL)$$

Step-by-Step Execution:

1. **Step 1 (Join):** Perform an equijoin or natural join between `STUDENT` and `ENROLL` on common attribute `SID`. This combines each student tuple with their corresponding course enrollment tuples. Students who have not enrolled in any course are naturally excluded.
2. **Step 2 (Projection):** Apply the projection operator (π) to project only the `SNAME` attribute, automatically eliminating duplicate student names from multiple course enrollments.

Problem 3: Library Management Relational Algebra Queries 10 MARKS END-SEM Q4.b

Given Schema:
`BOOK(Book_Id, Title, Author, Publisher, Price)`
`MEMBER(Member_Id, MName, Phone, Join_Date)`
`ISSUE_RECORD(Issue_Id, Member_Id, Book_Id, Issue_Date, Due_Date, Return_Date)`

Query (a): Retrieve all books written by author 'Navathe'.

$$\sigma_{Author = 'Navathe'}(BOOK)$$

To project specific attributes (Book_Id, Title):

$$\pi_{Book_Id,\ Title}(\sigma_{Author = 'Navathe'}(BOOK))$$

Query (b): List all overdue books (books where Return_Date is NULL and Due_Date < CURRENT_DATE).

$$\pi_{Book_Id,\ Title}(BOOK \bowtie (\sigma_{Return_Date = NULL \ \wedge\ Due_Date < CURRENT_DATE}(ISSUE_RECORD)))$$

Explanation: Filter `ISSUE_RECORD` for overdue records, then natural join with `BOOK` to retrieve titles and IDs.

Query (c): Find the names of members who haven't borrowed any book.

$$\pi_{MName}(MEMBER \bowtie (\pi_{Member_Id}(MEMBER) - \pi_{Member_Id}(ISSUE_RECORD)))$$

Step-by-Step Derivation:

1. Find all registered member IDs: *AllMembers* = π_{Member_Id}(MEMBER).
2. Find member IDs who borrowed at least one book: *Borrowers* = π_{Member_Id}(ISSUE_RECORD).
3. Calculate members who never borrowed using Set Difference (−): *NonBorrowers* = *AllMembers* − *Borrowers*.
4. Natural join *NonBorrowers* with `MEMBER` and project `MName`.

Query (d): Advanced — Find members who have borrowed ALL books written by 'Korth' (Division Operator ÷).

$$KorthBooks = \pi_{Book_Id}(\sigma_{Author = 'Korth'}(BOOK))$$
$$MemberBorrowings = \pi_{Member_Id,\ Book_Id}(ISSUE_RECORD)$$
$$ResultMembers = MemberBorrowings \div KorthBooks$$
$$FinalQuery = \pi_{MName}(MEMBER \bowtie ResultMembers)$$

3.2 Relational Schema Design & ER Mapping Case Study

Design of Relational Schema for Library Management System

10 MARKS END-SEM Q4.b

Following the formal 7-step ER-to-Relational mapping rules, the conceptual library ER diagram maps into the following normalized relational schema with primary and foreign key constraints:

```
-- 1. Table for BOOK entity
CREATE TABLE Book (
    Book_Id VARCHAR2(10) PRIMARY KEY,
    Title VARCHAR2(100) NOT NULL,
    Publisher_Id VARCHAR2(10) NOT NULL,
    Price NUMBER(7,2) CHECK (Price >= 0),
    Edition NUMBER(3),
    CONSTRAINT fk_book_pub FOREIGN KEY (Publisher_Id) REFERENCES Publisher(Publisher_Id)
);

-- 2. Table for multivalued attribute 'Author'
CREATE TABLE Book_Author (
    Book_Id VARCHAR2(10),
    Author_Name VARCHAR2(50),
    CONSTRAINT pk_book_author PRIMARY KEY (Book_Id, Author_Name),
    CONSTRAINT fk_author_book FOREIGN KEY (Book_Id) REFERENCES Book(Book_Id) ON DELETE CASCADE
);

-- 3. Table for PUBLISHER entity
CREATE TABLE Publisher (
    Publisher_Id VARCHAR2(10) PRIMARY KEY,
    Pub_Name VARCHAR2(50) NOT NULL,
    Address VARCHAR2(100),
    Phone VARCHAR2(15)
);

-- 4. Table for MEMBER entity
CREATE TABLE Member (
    Member_Id VARCHAR2(10) PRIMARY KEY,
    MName VARCHAR2(50) NOT NULL,
    Phone VARCHAR2(15) UNIQUE,
    Email VARCHAR2(100) UNIQUE,
    Join_Date DATE DEFAULT SYSDATE
);

-- 5. Table for M:N Relationship 'BORROWS / ISSUES'
CREATE TABLE Issue_Record (
    Issue_Id NUMBER(8) PRIMARY KEY,
    Member_Id VARCHAR2(10) NOT NULL,
```

```
    Book_Id VARCHAR2(10) NOT NULL,
    Issue_Date DATE DEFAULT SYSDATE,
    Due_Date DATE NOT NULL,
    Return_Date DATE,
    Fine_Amount NUMBER(6,2) DEFAULT 0.00,
    CONSTRAINT fk_issue_mem FOREIGN KEY (Member_Id) REFERENCES Member(Member_Id),
    CONSTRAINT fk_issue_book FOREIGN KEY (Book_Id) REFERENCES Book(Book_Id),
    CONSTRAINT chk_dates CHECK (Due_Date >= Issue_Date)
);
```

3.3 SQL Practical Queries & Master Scripts

Problem 1: Comprehensive Student Table DDL, DML & TCL Transaction

10 MARKS END-SEM Q4.c

Requirement: Write SQL queries for the following tasks on a `STUDENT` table:

i. Create the table with appropriate data types.

ii. Insert 2 sample records.

iii. Update a student's name based on ID.

iv. Delete a student record with a given ID.

v. Use a transaction to roll back a delete operation.

```
-- (i) Create the table with appropriate data types and integrity constraints
CREATE TABLE Student (
    RollNo NUMBER(6) PRIMARY KEY,
    Name VARCHAR2(50) NOT NULL,
    Branch VARCHAR2(30) DEFAULT 'Computer Engineering',
    Marks NUMBER(5,2) CHECK (Marks BETWEEN 0 AND 100),
    City VARCHAR2(30),
    MobileNumber VARCHAR2(15) UNIQUE,
    EmailId VARCHAR2(100) UNIQUE
);

-- (ii) Insert 2 sample records
INSERT INTO Student (RollNo, Name, Branch, Marks, City, MobileNumber, EmailId)
VALUES (101, 'Aarav Sharma', 'Computer Engineering', 92.50, 'Navi Mumbai', '9820123456',
'aarav.sharma@dypatil.edu');

INSERT INTO Student (RollNo, Name, Branch, Marks, City, MobileNumber, EmailId)
VALUES (102, 'Ananya Patel', 'Information Technology', 88.00, 'Pune', '9820654321',
'ananya.patel@dypatil.edu');

COMMIT; -- Commit the newly inserted records

-- (iii) Update a student's name based on ID
UPDATE Student
SET Name = 'Aarav S. Sharma'
WHERE RollNo = 101;

COMMIT;

-- (iv) & (v) Transaction demonstrating deletion and subsequent ROLLBACK
-- Start transaction block
SAVEPOINT before_delete;

-- Perform delete on student with RollNo = 102
DELETE FROM Student
WHERE RollNo = 102;

-- Verify deletion within session (RollNo 102 is temporarily removed)
SELECT * FROM Student WHERE RollNo = 102; -- Returns 0 rows

-- Rollback the delete operation to restore the record safely!
ROLLBACK TO before_delete;

-- Confirm restoration: RollNo 102 is completely restored!
SELECT RollNo, Name, Branch FROM Student WHERE RollNo = 102;
```

```
COMMIT; -- Finalize consistent transaction state
```

Problem 2: Student Branch Analytics, Aggregates & Pattern Matching 5 MARKS CA2 Sec A Q3 • Q7

Given Schema: `Student(RollNo, Name, Branch, Marks, City, MobileNumber, EmailId)`

Query (a): Count the total number of students in each branch.

```
SELECT Branch, COUNT(*) AS TotalStudents
FROM Student
GROUP BY Branch;
```

Query (b): Find the average marks branch-wise.

```
SELECT Branch, ROUND(AVG(Marks), 2) AS AverageMarks
FROM Student
GROUP BY Branch;
```

Query (c): Find the topper's marks in the entire class and display the student's details.

```
-- Subquery to find student details corresponding to the maximum marks
SELECT RollNo, Name, Branch, Marks
FROM Student
WHERE Marks = (SELECT MAX(Marks) FROM Student);
```

Query (d): Find the total marks scored by each branch.

```
SELECT Branch, SUM(Marks) AS TotalMarksScored
FROM Student
GROUP BY Branch;
```

Query (e): Display the overall average marks of the class.

```
SELECT ROUND(AVG(Marks), 2) AS ClassAverageMarks
FROM Student;
```

Query (f): Display the names of students whose name starts with 'A' and ends with 'E', having exactly 5 characters.

```
-- Wildcard '_' matches exactly one character; '%' matches 0 or more characters
SELECT RollNo, Name, Branch
FROM Student
WHERE UPPER(Name) LIKE 'A___E'; -- Exactly 3 underscores between A and E (Total 5 chars)
```

Problem 3: SQL Set Operations (UNION, UNION ALL, INTERSECT, MINUS) 5 MARKS CA2 Sec A Q4

Assume two tables: `CS_Students(RollNo, Name)` and `IT_Students(RollNo, Name)`.

```
-- 1. UNION: Combines records from both tables and removes duplicates
SELECT RollNo, Name FROM CS_Students
UNION
SELECT RollNo, Name FROM IT_Students;

-- 2. UNION ALL: Combines records without removing duplicates (faster)
SELECT RollNo, Name FROM CS_Students
UNION ALL
SELECT RollNo, Name FROM IT_Students;

-- 3. INTERSECT: Returns only students present in BOTH tables
SELECT RollNo, Name FROM CS_Students
INTERSECT
SELECT RollNo, Name FROM IT_Students;

-- 4. MINUS (EXCEPT in standard SQL): Returns CS students who are NOT in IT table
SELECT RollNo, Name FROM CS_Students
```

```
MINUS
SELECT RollNo, Name FROM IT_Students;
```

Problem 4: Advanced Joins, Views & Subqueries 5 MARKS CA2 Sec A Q5 • Q6

1. Outer Join Query: Find Students Not Enrolled in Any Course

```
SELECT S.RollNo, S.Name, S.Branch
FROM Student S
LEFT OUTER JOIN Student_Enrollment E ON S.RollNo = E.RollNo
WHERE E.CourseId IS NULL; -- Filters rows with no matching enrollment record
```

2. Self Join Query: Find Employees Earning More than Their Managers

```
SELECT E.EmpName AS EmployeeName, E.Salary AS EmpSalary,
       M.EmpName AS ManagerName, M.Salary AS MgrSalary
FROM Employee E
JOIN Employee M ON E.ManagerId = M.EmpId
WHERE E.Salary > M.Salary;
```

3. Subquery: Find the Second Highest Marks in the Entire Class

```
SELECT MAX(Marks) AS SecondHighestMarks
FROM Student
WHERE Marks < (SELECT MAX(Marks) FROM Student);
```

4. Correlated Subquery with EXISTS: Find Departments Having at Least One Student with Marks > 90

```
SELECT D.DeptId, D.DeptName
FROM Department D
WHERE EXISTS (
    SELECT 1 FROM Student S
    WHERE S.Branch = D.DeptName AND S.Marks > 90
);
```

3.4 Functional Dependencies & Normalization Solved Numericals

ALGORITHM 1: COMPUTING ATTRIBUTE CLOSURE (X^+)

Input: A set of attributes X and a set of functional dependencies F.

Output: The complete closure set X^+ under F.

1. Initialize $X^+ = X$.
2. Repeat until no more attributes can be added to X^+:
   For each functional dependency $Y \rightarrow Z$ in F:
   If $Y \subseteq X^+$, then set $X^+ = X^+ \cup Z$.
3. Return X^+.

ALGORITHM 2: FINDING CANDIDATE KEYS OF A RELATION

1. **Essential Attributes (Never on RHS):** Attributes that do not appear on the right-hand side of any FD in F *must* be part of every candidate key. Let this set be E.
2. Compute the closure E^+. If E^+ contains all attributes of R, then E is the unique Candidate Key!
3. If E^+ does not contain all attributes, combine E with other attributes (starting with combinations of size 1, then 2, etc.) until the closure covers all attributes of R. Ensure minimality (no proper subset must be a candidate key).

Numerical 1: Attribute Closure & Candidate Keys

5 MARKS END-SEM Q3.d

Problem: Given a relation $R(A, B, C)$ with functional dependencies:
F = { A → B, B → C }
Find the closure of A (A^+), list all candidate keys, and determine the highest normal form of R.

Step 1: Finding the Closure of A (A^+)

1. Initialize: A^+ = {A}.
2. Apply $A \rightarrow B$: Since {A} ⊆ A^+, add B → A^+ = {A, B}.
3. Apply $B \rightarrow C$: Since {B} ⊆ A^+, add C → A^+ = {A, B, C}.

Result: A^+ = { A, B, C }

Step 2: Listing All Candidate Keys

- Notice that attribute A never appears on the right-hand side of any dependency in F. Therefore, A *must* be present in every candidate key.
- Since A^+ = {A, B, C} covers all attributes of relation R, and A cannot be reduced further, **{A} is the sole Candidate Key**.
- Candidate Key: **{A}**. Prime attribute: **{A}**; Non-prime attributes: **{B, C}**.

Step 3: Determining the Highest Normal Form

- **1NF Check:** All attributes are atomic → **Satisfied**.
- **2NF Check:** Candidate key is a single attribute {A}. Partial dependencies can only exist if the candidate key is composite. Since there is no composite key, partial dependencies are impossible → **In 2NF**.
- **3NF Check:** Consider the dependency $B \rightarrow C$:
  - Is B a super key? No (B^+ = {B, C} ≠ {A, B, C}).
  - Is C a prime attribute? No (C is not part of any candidate key).
  Both conditions fail! $B \rightarrow C$ is a **transitive dependency** ($A \rightarrow B$ and $B \rightarrow C$).
  Therefore, R is **NOT in 3NF**.
- **Conclusion:** The highest normal form of R is **2NF**.

Numerical 2: Converting Relation into BCNF Step-by-Step

10 MARKS CA2 Sec B Q7

Problem: Convert the given relation into BCNF:
R(A, B, C, D, E) with FDs: F = { A → B, C → D, AC → E }

Step 1: Determine the Candidate Key(s) of R

- Attributes appearing on RHS: {B, D, E}.
- Attributes **never on RHS**: {A, C}.
- Therefore, {A, C} must be part of every candidate key. Let us compute $(AC)^+$:
  $(AC)^+ = \{A, C\}$
  Apply $A \rightarrow B \rightarrow (AC)^+ = \{A, B, C\}$
  Apply $C \rightarrow D \rightarrow (AC)^+ = \{A, B, C, D\}$
  Apply $AC \rightarrow E \rightarrow (AC)^+ = \{A, B, C, D, E\} = R$.
- Since $(AC)^+$ contains all attributes of R, the unique **Candidate Key is {A, C}**.

Step 2: Check BCNF Violation for Each Dependency

A dependency $X \rightarrow Y$ satisfies BCNF if and only if X is a super key.

1. **FD 1: $A \rightarrow B$:**
   $A^+ = \{A, B\} \neq R$. A is **NOT a super key**. → **Violates BCNF!**
2. **FD 2: $C \rightarrow D$:**
   $C^+ = \{C, D\} \neq R$. C is **NOT a super key**. → **Violates BCNF!**
3. **FD 3: $AC \rightarrow E$:**
   $(AC)^+ = \{A, B, C, D, E\} = R$. AC is a super key. → **Satisfies BCNF**.

Step 3: Systematic BCNF Decomposition

We decompose R along the violating dependency $A \rightarrow B$:

- **R_1(A, B)** with FD: { A → B }.
  In R_1, $A^+ = \{A, B\} = R_1$. Here A is a super key! → **R_1 is in BCNF**.
- **R_{rem}(A, C, D, E)** (attributes remaining: R − {B}).
  The FDs applicable to R_{rem} are { C → D, AC → E }.
  The candidate key of R_{rem} is {A, C}.

Now inspect R_{rem}(A, C, D, E): The dependency $C \rightarrow D$ violates BCNF because C is not a super key of R_{rem}. Decompose R_{rem} along $C \rightarrow D$:

- **R_2(C, D)** with FD: { C → D }.
  In R_2, $C^+ = \{C, D\} = R_2$. C is a super key! → **R_2 is in BCNF**.
- **R_3(A, C, E)** (attributes remaining: R_{rem} − {D}).
  The FD applicable to R_3 is { AC → E }.
  In R_3, $(AC)^+ = \{A, C, E\} = R_3$. AC is a super key! → **R_3 is in BCNF**.

Final Resulting Schemas

FINAL BCNF RELATIONS

1. **R_1(A, B)** — Primary Key: A
2. **R_2(C, D)** — Primary Key: C
3. **R_3(A, C, E)** — Primary Key: {A, C} (with Foreign Keys referencing R_1 and R_2)

Verification of Decomposition Properties

- **Lossless Join Verification:**
  - $R_2 \cap R_3 = \{C, D\} \cap \{A, C, E\} = \{C\}$. Since $C \rightarrow D$ holds, {C} is a key for R_2. Thus $R_2 \bowtie R_3$ is lossless (= R_{rem}).
  - $R_1 \cap R_{rem} = \{A, B\} \cap \{A, C, D, E\} = \{A\}$. Since $A \rightarrow B$ holds, {A} is a key for R_1. Thus $R_1 \bowtie R_{rem}$ is lossless (= R).
  - Therefore, the entire decomposition is strictly **Lossless**!
- **Dependency Preservation Verification:**
  - $A \rightarrow B$ is preserved in R_1.

- $C \rightarrow D$ is preserved in R_2.
- $AC \rightarrow E$ is preserved in R_3.
- All functional dependencies in F are fully preserved without cross-table joins!

Numerical 3: Testing Lossless-Join Decomposition Condition

5 MARKS END-SEM Q1.d

Problem: Given relation $R(A, B, C)$ with $F = \{ A \rightarrow B \}$.
Check whether decomposition into $D_1 = \{ R_1(A, B), R_2(B, C) \}$ and $D_2 = \{ R_1(A, B), R_3(A, C) \}$ is lossless or lossy.

Case 1: Decomposition $D_1 = \{ R_1(A, B), R_2(B, C) \}$

- Common attributes: $R_1 \cap R_2 = \{B\}$.
- Compute closure of common attributes: $B^+ = \{B\}$.
- Does $B^+ \supseteq R_1$? No ($\{B\} \not\supseteq \{A, B\}$).
- Does $B^+ \supseteq R_2$? No ($\{B\} \not\supseteq \{B, C\}$).
- **Conclusion for D_1:** The common attribute B is NOT a super key for either relation. Hence, **D_1 is a LOSSY decomposition**! Rejoining produces spurious tuples.

Case 2: Decomposition $D_2 = \{ R_1(A, B), R_3(A, C) \}$

- Common attributes: $R_1 \cap R_3 = \{A\}$.
- Compute closure of common attributes: $A^+ = \{A, B\}$ (using $A \rightarrow B$).
- Does $A^+ \supseteq R_1$? Yes! $A^+ = \{A, B\} \supseteq R_1$.
- **Conclusion for D_2:** Since the common attribute A functionally determines R_1, **D_2 is strictly a LOSSLESS-JOIN decomposition**!

3.5 PL/SQL Complete Code Implementations

Problem 1: Stored Procedure with Audit Logging Trigger 10 MARKS END-SEM Q5.b

Requirement: Design a stored procedure in PL/SQL that accepts employee details, inserts them into an employee table, and uses a trigger to log the insertion into an audit table. Explain how your design ensures data integrity and auditing.

Step 1: Supporting Table Definitions

```
-- Base Employee Table
CREATE TABLE Employee (
    EmpId NUMBER(6) PRIMARY KEY,
    EmpName VARCHAR2(50) NOT NULL,
    Department VARCHAR2(30) NOT NULL,
    Salary NUMBER(10,2) CHECK (Salary > 0),
    HireDate DATE DEFAULT SYSDATE
);

-- Audit Table for Tracking Insertions
CREATE TABLE Employee_Audit (
    AuditId NUMBER(8) GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    EmpId NUMBER(6) NOT NULL,
    Action VARCHAR2(20) DEFAULT 'INSERT',
    ChangedBy VARCHAR2(50),
    ChangeTimestamp TIMESTAMP DEFAULT SYSTIMESTAMP,
    InsertedSalary NUMBER(10,2)
);
```

Step 2: Designing the Audit Trigger (AFTER INSERT ON Employee FOR EACH ROW)

```
CREATE OR REPLACE TRIGGER trg_emp_insert_audit
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
    -- Log every successfully inserted row into the audit table
    INSERT INTO Employee_Audit (
        EmpId,
        Action,
        ChangedBy,
        ChangeTimestamp,
        InsertedSalary
    ) VALUES (
        :NEW.EmpId,
        'INSERT',
        USER, -- Oracle built-in returning current session username
        SYSTIMESTAMP,
        :NEW.Salary
    );
END;
/
```

Step 3: Designing the Stored Procedure (hire_employee)

```
CREATE OR REPLACE PROCEDURE hire_employee (
    p_empid      IN Employee.EmpId%TYPE,
    p_empname    IN Employee.EmpName%TYPE,
    p_dept       IN Employee.Department%TYPE,
    p_salary     IN Employee.Salary%TYPE
)
IS
    -- User-defined exception for invalid salary
    ex_invalid_salary EXCEPTION;
BEGIN
    -- Input Validation for Data Integrity
    IF p_salary <= 0 OR p_salary IS NULL THEN
        RAISE ex_invalid_salary;
    END IF;
```

```
    -- Insert into base table (Automatically fires the audit trigger!)
    INSERT INTO Employee (EmpId, EmpName, Department, Salary, HireDate)
    VALUES (p_empid, p_empname, p_dept, p_salary, SYSDATE);

    -- Commit transaction: commits both employee row and audit record atomically!
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Success: Employee ' || p_empid || ' hired and logged successfully.');

EXCEPTION
    WHEN ex_invalid_salary THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Salary must be greater than zero.');
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Employee ID ' || p_empid || ' already exists.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('System Error: ' || SQLERRM);
END hire_employee;
/
```

Step 4: Verification & Testing Block

```
BEGIN
    hire_employee(201, 'Neha Deshmukh', 'Research', 75000);
END;
/
-- Verification Queries:
SELECT * FROM Employee WHERE EmpId = 201;
SELECT * FROM Employee_Audit WHERE EmpId = 201;
```

Architectural Analysis: Data Integrity & Auditing Guarantees

- **Non-Bypassable Auditing:** Because the trigger resides at the database engine level, an audit row is created regardless of whether the insertion originates from the stored procedure, an interactive SQL prompt, a Python script, or a web frontend.
- **Transactional Atomicity:** The base insert and audit log insert participate in the *same atomic database transaction*. If the trigger fails, the main insert rolls back automatically, preventing orphaned or unlogged records.

Problem 2: Explicit Cursor Processing with Attributes 5 MARKS

Requirement: Write a PL/SQL block using an explicit cursor to increase the salary of all employees in the 'IT' department by 10%, displaying their details and total rows updated.

```
DECLARE
    -- 1. Declare explicit cursor
    CURSOR cur_it_emp IS
        SELECT EmpId, EmpName, Salary
        FROM Employee
        WHERE Department = 'IT'
        FOR UPDATE OF Salary; -- Locks selected rows for update

    -- Anchored variables
    v_id   Employee.EmpId%TYPE;
    v_name Employee.EmpName%TYPE;
    v_sal  Employee.Salary%TYPE;
BEGIN
    -- 2. Open cursor
    OPEN cur_it_emp;

    LOOP
        -- 3. Fetch row into variables
        FETCH cur_it_emp INTO v_id, v_name, v_sal;

        -- Check %NOTFOUND attribute to exit when all rows processed
        EXIT WHEN cur_it_emp%NOTFOUND;
```

```
        -- Procedural update using CURRENT OF cursor clause
        UPDATE Employee
        SET Salary = Salary * 1.10
        WHERE CURRENT OF cur_it_emp;

        DBMS_OUTPUT.PUT_LINE('Updated Emp ' || v_id || ' (' || v_name || ') Old: ' || v_sal || ' New: '
|| (v_sal * 1.10));
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Total IT Employees Updated: ' || cur_it_emp%ROWCOUNT);

    -- 4. Close cursor
    CLOSE cur_it_emp;
    COMMIT;
END;
/
```

Problem 3: Weekend DML Restriction Trigger (BEFORE DELETE/UPDATE) 5 MARKS

Requirement: Write a database trigger to prevent any `DELETE` or `UPDATE` operations on the `Employee` table during weekends (Saturday and Sunday).

```
CREATE OR REPLACE TRIGGER trg_restrict_weekend_ops
BEFORE DELETE OR UPDATE ON Employee
DECLARE
    v_day VARCHAR2(10);
BEGIN
    -- Extract current 3-character day abbreviation in uppercase
    v_day := UPPER(TO_CHAR(SYSDATE, 'DY'));

    IF v_day IN ('SAT', 'SUN') THEN
        -- Raise standard application error and abort the DML operation
        RAISE_APPLICATION_ERROR(-20001, 'Security Violation: Database modifications are prohibited on
weekends.');
    END IF;
END;
/
```

Problem 4: PL/SQL Stored Function with Return Value & Exception Handling 5 MARKS

Requirement: Write a PL/SQL stored function `calculate_library_fine` that accepts a book's due date and actual return date, calculates the total fine based on tiered rates (₹2/day for first 7 days, ₹5/day thereafter), and handles invalid input dates cleanly.

```
CREATE OR REPLACE FUNCTION calculate_library_fine (
    p_due_date    IN DATE,
    p_return_date IN DATE DEFAULT SYSDATE
)
RETURN NUMBER
IS
    v_overdue_days NUMBER := 0;
    v_total_fine   NUMBER(8,2) := 0;
    ex_negative_days EXCEPTION;
BEGIN
    -- Check for return before or on due date
    IF p_return_date <= p_due_date THEN
        RETURN 0;
    END IF;

    -- Calculate total overdue days (ignoring time components)
    v_overdue_days := TRUNC(p_return_date) - TRUNC(p_due_date);

    IF v_overdue_days < 0 THEN
        RAISE ex_negative_days;
    END IF;

    -- Tiered fine structure:
    -- Days 1 to 7: ₹2.00 per day
```

```
    -- Days 8 and beyond: ₹5.00 per day
    IF v_overdue_days <= 7 THEN
        v_total_fine := v_overdue_days * 2.00;
    ELSE
        v_total_fine := (7 * 2.00) + ((v_overdue_days - 7) * 5.00);
    END IF;

    RETURN v_total_fine;

EXCEPTION
    WHEN ex_negative_days THEN
        DBMS_OUTPUT.PUT_LINE('Error: Invalid dates — return date precedes due date.');
        RETURN -1;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Calculation Error: ' || SQLERRM);
        RETURN -1;
END calculate_library_fine;
/

-- Verification query demonstrating calling the stored function inside SQL:
SELECT IssueId, BookId, MemberId, DueDate, ReturnDate,
       calculate_library_fine(DueDate, ReturnDate) AS ComputedFine
FROM IssueRecord
WHERE ReturnDate > DueDate;
```

3.6 Transactions & Serializability Solved Numericals

Problem 1: Demonstrating Conflict Serializability Using Precedence Graph

5 MARKS END-SEM Q2.f

Problem: Given two transactions T_1 and T_2 with the following interleaved schedule S_1:

S_1: $R_1(A)$; $W_1(A)$; $R_2(A)$; $W_2(A)$; $R_2(B)$; $W_2(B)$; $R_1(B)$; $W_1(B)$

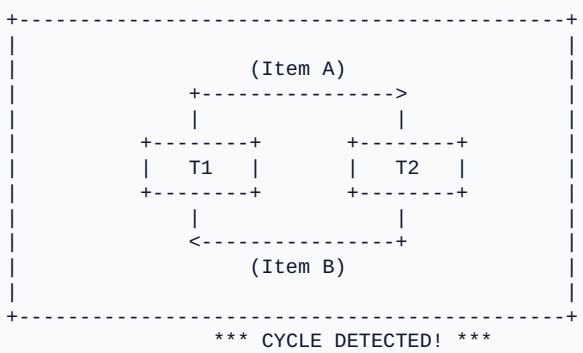
Demonstrate whether the schedule is conflict-serializable using a precedence graph.

Step 1: Identify Conflicting Operations

Operations conflict if they belong to different transactions, access the same data item, and at least one is a write.

1. **On Data Item A:**
   - $W_1(A)$ occurs before $R_2(A)$ → Conflict! Creates a directed dependency: $\mathbf{T_1 \rightarrow T_2}$.
   - $W_1(A)$ occurs before $W_2(A)$ → Conflict! Consistent with $\mathbf{T_1 \rightarrow T_2}$.
2. **On Data Item B:**
   - $W_2(B)$ occurs before $R_1(B)$ → Conflict! Creates a directed dependency: $\mathbf{T_2 \rightarrow T_1}$.
   - $W_2(B)$ occurs before $W_1(B)$ → Conflict! Consistent with $\mathbf{T_2 \rightarrow T_1}$.

Step 2: Construct the Precedence Graph



Step 3: Cycle Detection & Conclusion

- The graph contains a directed edge $T_1 \rightarrow T_2$ (due to conflict on A) and a return directed edge $T_2 \rightarrow T_1$ (due to conflict on B).
- This forms a directed cycle: $\mathbf{T_1 \rightarrow T_2 \rightarrow T_1}$.

- **Definitive Conclusion:** Because the precedence graph contains a cycle, the schedule S_1 is **NOT Conflict Serializable**! No equivalent serial schedule exists, and executing this schedule may result in data inconsistency.

Problem 2: Conflict-Serializable Schedule & Equivalent Serial Order 5 MARKS

Problem: Consider schedule S_2 over three transactions T_1, T_2, T_3:

S_2: $R_1(X)$; $W_1(X)$; $R_2(X)$; $W_2(X)$; $R_3(Y)$; $W_3(Y)$; $R_1(Y)$; $W_1(Y)$

Test if S_2 is conflict-serializable and find the equivalent serial schedule if it is.

Step 1: Trace All Conflicting Pairs

1. On item X: $W_1(X)$ precedes $R_2(X)$ and $W_2(X)$ → Directed edge: $\mathbf{T_1 \rightarrow T_2}$.
2. On item Y: $W_3(Y)$ precedes $R_1(Y)$ and $W_1(Y)$ → Directed edge: $\mathbf{T_3 \rightarrow T_1}$.
3. Between T_2 and T_3: No conflicting operations on any shared data item → No edge.

Step 2: Precedence Graph & Topological Sort



- The graph contains **NO CYCLES**.
- **Conclusion:** Schedule S_2 is strictly **Conflict Serializable**!
- Applying Topological Sort on the graph yields the unique equivalent serial schedule: **Equivalent Serial Schedule: $\mathbf{T_3 \rightarrow T_1 \rightarrow T_2}$.**

Problem 3: Blind Write Schedule — View Serializable but NOT Conflict Serializable 5 MARKS

Given Schedule S_3:

S_3: $R_1(A)$; $W_2(A)$; $W_1(A)$; $W_3(A)$

(Notice that T_2 and T_3 perform *Blind Writes* — writing without reading A first).

Analysis:

- **Conflict Serializability Check:**
  - $R_1(A)$ precedes $W_2(A)$ ⇒ $T_1 \rightarrow T_2$.
  - $W_2(A)$ precedes $W_1(A)$ ⇒ $T_2 \rightarrow T_1$.

  Graph contains cycle $T_1 \rightarrow T_2 \rightarrow T_1$ ⇒ **NOT Conflict Serializable!**
- **View Serializability Check:**

  Compare S_3 with serial schedule S' = $T_1 \rightarrow T_2 \rightarrow T_3$:
  1. **Initial read of A:** Performed by T_1 in both schedules ⇒ Satisfied.
  2. **Updated reads:** None (no transaction reads an updated value in either schedule) ⇒ Trivially satisfied.
  3. **Final write of A:** Performed by T_3 in both schedules ⇒ Satisfied.
- **Conclusion:** Schedule S_3 is **View Serializable** (view equivalent to serial schedule $T_1 \rightarrow T_2 \rightarrow T_3$), even though it is **NOT Conflict Serializable**!